REQUIREMENTS SPECIFICATION

Zümm

Beekeeping Management and Monitoring System

Project	Beekeeping management application (J2EE mini-project)
Type	Functional and technical requirements specification
Version	1.0
Date	July 13, 2026
Method	Manager-GO – Drawing up a requirements specification

Connected hive & performance dashboards

Document following the standard outline: Context / Objectives / Scope / Functional description of needs / Budget envelope / Schedule

Contents

Version history

Version	Date	Author	Changes
0.1	July 9, 2026	Project team	Initial structure and transcription of the brief.
0.5	July 11, 2026	Project team	Beekeeping domain reference and illustrations.
1.0	July 13, 2026	Project team	Market features, defect-prevention requirements, data dictionary, use cases, UML design and test plan.

CHAPTER 1

Context and problem definition

1.1 General overview

The **Zümm** project aims to design an information system dedicated to the management and monitoring of beekeeping activities. Modern beekeeping relies on the regular monitoring of numerous hives, often spread across several geographic sites and operated by different beekeepers. Tracking colony health, productivity and maintenance operations represents a significant organisational burden, today handled in a largely manual and fragmented way.

1.2 Problem statement

Without a centralised tool, beekeeping operators face several difficulties:

- no structured traceability of hive visits and inspections
- difficulty in planning and coordinating the work of beekeeping agents
- lack of visibility over production (honey, weight) and over the profitability of each site or hive
- late detection of anomalies (population decline, production drop, unplanned opening of a hive)
- inability to automatically process measurements from heterogeneous sensors.

1.3 Purpose

The system must provide a structured response in two complementary parts:

1. a **manual management** of handling operations, their planning and monitoring (first part, the subject of this project)
2. a **remote, automated supervision** of performance indicators, fed by sensors, intended to be developed as part of a later project but whose integration the system must anticipate.

CHAPTER 2

Project objectives

2.1 Main objective

Provide a **multi-tier, scalable and loosely coupled** application that manages the entire life cycle of hives, from their physical composition to production monitoring, and offers decision-support dashboards to the various user profiles.

2.2 Specific objectives

- Ensure **CRUD** operations (create, read, update, delete) on all business entities.
- Manage the *farmer / farm / site / hive / body or super / frame* hierarchy.
- Plan, approve and track the inspection visits of beekeeping agents.
- Produce structured and actionable **visit reports**.
- Offer filterable **dashboards** (agents × hives calendar, production alerts, health alerts, financial summary).
- Anticipate the integration of **time-stamped and geolocated** measurements from sensors with heterogeneous units.
- Guarantee **internationalisation**, **security** and **interoperability** (web services).

2.3 Expected results (success indicators)

- Support for at least **three languages** (FR, EN, AR) in the demonstration version.
- Automatic detection of **configurable** threshold breaches (production, population).
- Presentation of production data by farm, site and hive as charts.
- Exposure of an API that can be queried by third-party applications (Excel, etc.).

CHAPTER 3

Project scope

3.1 Covered domain

The scope of this requirements specification covers **the first part** of the system: the manual management of operations, their planning and monitoring, as well as the preparation of the architecture for the future integration of automated indicators.

3.2 Business model and management rules

Entity	Management rules
Farmer	May own several farms.
Farm	May include several apiary sites.
Site	Contains one or more hives, has a geographic location (latitude, longitude, altitude, commissioning date, possible relocation / closure dates). A site may host hives belonging to different farmers and different farms.
Hive	Made up of a base compartment (base / body) and at most **5** extension levels (supers).
Body / Super	May hold wax frames installed on identified *slots*. The base/body must contain **at least 1** frame; the maximum capacity of a body, as of a super, is **10 frames**.
Frame	Occupies a single slot in the body or the super.
Beekeeping agent	Carries out the visits and inspections. A hive may be monitored successively by several agents, but it is under the responsibility of **a single agent** at any given time.
Supervising agent	Approves the visit schedule of the hives under their charge.

Composition constraint: 1 hive = 1 mandatory base/body + 0 to 5 supers, each body/super = 1 to 10 frames, one frame per slot.

3.3 Users and access rights

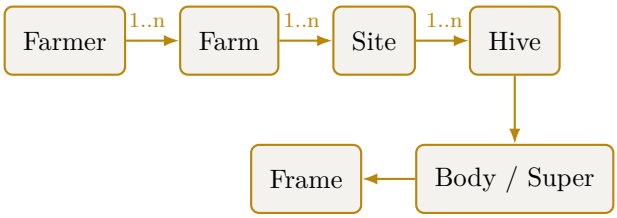

Figure 3.1. Hierarchy of the Zümm system's business entities.

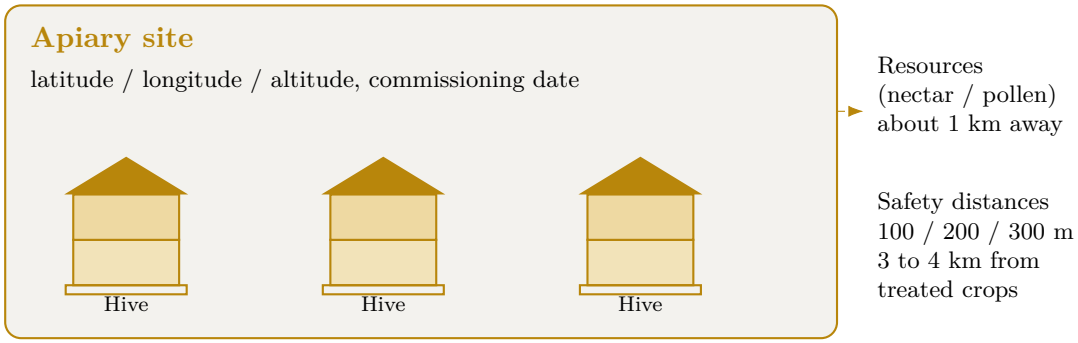

Figure 3.2. Diagram of an apiary site: several geolocated hives and placement constraints.

Profile	Role and permissions
Farmer / Owner	Views the production, profitability and summary dashboards; decides on closing a site or relocating hives.
Beekeeping agent	Carries out the visits, fills in the reports, performs operations on the hives under their responsibility.
Supervising agent	Approves the visit schedules of the hives they are responsible for.
Manager	Monitors the alerts (production or population drop) and triggers imminent inspections.
Administrator	Manages the system parameters, units of measurement, thresholds and reference data.

3.4 Out of scope

- The physical acquisition and hardware deployment of sensors (later project).
- Real-time automation of indicator collection (interface planned but not implemented in this phase).

CHAPTER 4

Functional description of needs

4.1 Entity management (CRUD)

The system must ensure the creation, viewing, modification and deletion of all business entities: farmers, farms, sites, hives, bodies/supers, frames, agents, visits and reports, in compliance with the management rules stated in chapter 3.

4.2 Planning and monitoring of visits

- Visits are **regular or irregular**, for various reasons: health inspection, populating with a new swarm, requeening, food replenishment, medication prescription, swarm splitting, adding a frame and/or an extension level, collecting honey frames, production assessment, assessment of the swarm's population and volume.
- Each agent follows a **visit schedule approved** by the supervising agent of the hive concerned.

4.2.1 Visit report

Each visit gives rise to a report containing: **date, time, duration, reason, findings, planned actions, actions carried out, recommendations**, a qualitative assessment of the swarm's population / volume, its health status and its productivity level on a **scale of 1 to 3**.

4.3 Dashboards

4.3.1 Calendar / agents × hives matrix

A matrix cross-referencing **agents and hives**, broken down by month, week, season and year, with **filters** and groupings by site, by agent and by responsible agent. Clicking a cell representing a planned visit displays a **pop-up** summarising what is planned (date, time, reason, actions) and, if the visit has already been carried out, what was done together with the actual date and time.

4.3.2 Production dashboard (farmer / owner)

Highlights the **site × hive** pairs whose production (weight) has exceeded a **configurable threshold**, indicating either an imminent honey harvest or an extension and/or swarm-splitting operation to encourage colony expansion.

4.3.3 Alerts dashboard (manager)

Flags the **site** × **hive** pairs whose production or population is dropping unusually beyond a configurable threshold, triggering an **alert** and an imminent inspection.

4.3.4 Summary dashboard (owner)

Charts, curves and histograms representing:

- the quantity produced per farm and per site
- the number of operations per farm, per site and per hive
- the **return on investment** (cost / production ratio) to decide the fate of each site or hive (closure, relocation).

4.4 Performance indicators (automated part: preparation)

The measurements are **time-stamped and geolocated**, expressed in **configurable** units (internationalisation of unit systems). The measurements of a single indicator do not necessarily share the same unit, as the sensors are heterogeneous. Indicators concerned:

- weight of the frame, the base and the extension level
- temperature and humidity, inside and outside the hive
- count of bee entry and exit movements
- noise / buzzing level, inside and outside
- light level, inside and outside
- wind speed and direction outside
- hive opening status (operation, storm, theft, animal attack, etc.).

4.4.1 Predictive analysis and connected monitoring

Connected monitoring solutions (precision beekeeping) such as Arnia, BroodMinder or BeeHero show that advanced exploitation of the same indicators makes it possible to anticipate colony events. The data model must remain open to these processes, planned for the automated part.

- acoustic detection of swarming 1 to 5 days in advance through analysis of the colony's sound signature
- confirmation of the queen's presence from the buzzing
- detection of opening, shock or tipping of the hive by accelerometer (theft, fall, animal attack), in addition to the opening indicator
- hive orientation by electronic compass
- continuous weighing by scale to track the nectar flow and production dynamics
- cloud-based analysis by artificial intelligence of acoustic signatures to correlate sound and hive activity.

4.4.2 Measurement ingestion protocol

The automated part remains out of the development scope, but the ingestion interface is specified now so that the data model and the architecture anticipate it.

- **Transport**: a field gateway transmits the measurements to the server, chosen according to connectivity, either by **REST over HTTPS** (`POST /api/mesures`) or by **MQTT** (publishing on the topic `zumm/{rucheId}/{indicateur}`).
- **Format**: **JSON**, a measurement carrying the hive identifier, the indicator type, the value, the unit, the timestamp and the geolocation. A batch upload is an array of measurements.
- **Frequency**: configurable per indicator (for example weight every 15 min, temperature every 5 min, acoustics sampled continuously), in order to control the data volume.
- **Robustness**: **asynchronous** ingestion via a queue, with **idempotency** on the key (hive, indicator, timestamp) to replay without duplicates. In a remote area, local buffering on the gateway then deferred synchronisation.
- **Security**: authenticated gateway (key or certificate), TLS channel. Rejection of values outside the physically plausible range (safeguard).

```
POST /api/mesures        Content-Type: application/json
{
  "rucheId": 42, "typeIndicateur": "TEMPERATURE_INTERNE",
  "valeur": 34.8, "unite": "degC",
  "horodatage": "2026-07-09T14:05:00Z",
  "latitude": 36.806, "longitude": 10.181
}
```

4.4.3 Default thresholds and alert logic

The thresholds are stored in the `SEUIL` table (configurable, chapter 5). The values below are **reference default values**, anchored in the beekeeping protocol (chapter 12).

Indicator	Default threshold	Alert triggered
Internal temperature (brood)	$< 32\,°C$ or $> 36\,°C$	Thermal risk to the brood
Internal humidity	$> 80\,\%$	Excess humidity, health risk
Weight: super gain	> production threshold	Imminent honey harvest
Weight: sudden drop	$> 1.5\,kg$ in less than $1\,h$	Swarming, theft, fall or attack
Production or population	Relative drop $> 20\,\%$ vs previous period	Imminent health inspection
Entry/exit activity	≈ 0 on a favourable day	Weakened colony or missing queen
Opening status	Open outside a planned visit	Intrusion or disturbance
Acoustic signature	Swarming pattern detected	Swarming pre-alert (1 to 5 days)

Evaluation rule: at each measurement, the value is converted to the reference unit (conversion formula, §5.3), then compared with the configurable threshold. To limit *false alerts*, an alert is only raised if the breach **persists over N consecutive measurements** (debounce / hysteresis).

Any alert remains a **decision-support aid** confirmed by the visit report: the agent keeps the final validation.

4.5 Summary grid of functions

Function	Description	Priority
Entity CRUD	Complete management of the business model	High
Visit scheduling	Planning + approval by the supervisor	High
Visit reports	Structured recording of findings and actions	High
Agents×hives calendar	Filterable matrix with summary pop-up	High
Production alerts	Detection of threshold breaches	High
Health alerts	Detection of abnormal population/production drops	High
Summary & ROI	Cost/production charts, decision support	Medium
Sensor indicators	Data model ready for automation	Medium
Defect-prevention requirements (drawn from the limits of existing solutions)		
Autonomous deployment	J2EE installation with no mandatory subscription or third-party service	High
Offline mode	Data entry without connection and deferred synchronisation	High
Simple authentication	Google OAuth and fast field-entry flow	High
Configurable modularity	Enabling modules per profile via ConfigZumm.ini	Medium
Data portability	CSV and TXT export, web-service API and backup, with no lock-in	High
Contextual help	Simple, consistent interface, user guidance	Medium
Web and mobile parity	Same functions on all devices	Medium
Internationalisation	FR, EN and AR XML file, extensible to other languages	High
Hardware openness	Heterogeneous sensors and configurable units	Medium
Asynchronous ingestion	Deferred time-stamped measurements, autonomous manual part	Medium
Human validation	Configurable thresholds and confirmation by the visit report	High
Exchange security	SSL and X.509 certificates, authenticated access	High
Decision support	Indicative alerts, the agent keeps the final validation	Medium

4.6 Additional features inspired by market solutions

In order to strengthen usability and value in use, the system draws on the best practices of reference beekeeping applications such as HiveTracks. These features extend the needs already described without replacing them.

Function	Description	Priority
Inspection photos	Attach one or more photos to each visit report for a visual follow-up of the colony and the frames.	High
Assisted voice entry	Dictate the visit report, the transcription automatically feeding the report fields.	Low
Weather context	Display the local weekly weather from the site location to inform operation decisions.	Medium
Map and foraging radii	Position the hives on a map and draw foraging radii (for example 1, 2 and 3 km, configurable units) to visualise the environment accessible to the bees.	Medium
Task list and reminders	Manage a task list and a reminder calendar (feeding, inspection, queen status) so as not to miss any deadline.	High
Queen tracking	Record the history of the queen's status (presence, age, replacement, marking) across visits.	High
Harvest and QR code	Record honey harvests and generate a QR code per batch presenting the tasting notes, the provenance and the photos, for traceability and valorisation.	Medium
Multi-device access	Offer web access and an experience suited to mobile terminals for field entry.	Medium

Consistency with the requirements: the foraging radii and the weather reuse the geolocation of the sites and the configurable-units mechanism. The reminders rely on the existing visit schedule, and the queen tracking enriches the visit report already defined.

4.7 Advanced management functions (inspiration from Apiary Book and BeePlus)

The beekeeping management applications Apiary Book and BeePlus emphasise organisation and traceability functions that complement the system's scope.

Function	Description	Priority
Offline mode and synchronisation	Enter field data without a connection and synchronise it automatically when the network returns.	High
Treatment tracking	Keep the history of the medications and health treatments applied to each hive.	High
Equipment and inventory management	Track the equipment (bodies, supers, frames, hives) and its allocation to sites.	Medium
Financial tracking	Record costs and revenues per farm, site and hive, in support of the return-on-investment calculation.	Medium
Data export	Export the records in CSV and TXT formats for external analysis.	Medium
Disease detection	Help identify diseases from the visit findings.	Low

Function	Description	Priority
Data backup	Back up and restore the data in the cloud.	Medium

Consistency with the requirements: the financial tracking feeds the return-on-investment dashboard, the CSV export extends the web-service API, and the treatment tracking enriches the medication-prescription visit reason.

4.8 Known limits of existing solutions and requirements to avoid them

The analysis of user feedback on market solutions (HiveTracks, Apiary Book, BeePlus) and on connected monitoring systems (Arnia, BroodMinder, BeeHero) brings out recurring limits. The following table sets each limit against the requirement adopted to avoid it in Zümm.

Observed limit	Requirement adopted to avoid it
Paid subscription and paywall	Self-deployable J2EE system, with no mandatory subscription; all management functions remain accessible.
Limited offline operation	Robust offline mode with deferred synchronisation; field entry does not require a permanent connection.
Forced account creation and friction	Simple authentication via Google OAuth and a fast entry flow; usability comes first.
Overload of useless features	Modules enabled per profile and via the ConfigZumm.ini configuration file, thanks to the loosely coupled architecture.
Dependency and data loss	Data controlled by the user, CSV and TXT export, web-service API and backup, with no proprietary lock-in.
Dense interface and learning curve	Simple, consistent and user-friendly interface with contextual help, in line with the usability requirements.
Uneven functions across platforms	Functional parity between web and mobile access thanks to the multi-tier architecture.
English-only solutions	Internationalisation through an XML file, at least FR, EN and AR, open to other languages.
High cost of sensor hardware	Openness to heterogeneous and low-cost sensors with configurable units, with no hardware lock-in.
Autonomy and connectivity in a remote area	Asynchronous ingestion of time-stamped measurements; the manual part works without sensors.
False alerts and sensor reliability	Configurable thresholds and human validation through the visit report; alerts remain a decision-support aid.
Data confidentiality and security	SSL encryption and X.509 certificates, authenticated access.
Over-reliance on automation	Positioned as decision support; the beekeeping agent keeps the final validation.

CHAPTER 5

Data dictionary and calculation rules

This chapter answers the question of the system's inputs and outputs. It defines the data handled, their types and the formulas used to compute the results.

5.1 Type conventions

The types used are: integer, decimal, text, date, time, datetime, boolean, enumeration and reference (foreign key to another entity). Physical quantities are associated with a configurable unit to allow the internationalisation of measurement systems.

5.2 Data dictionary (inputs)

Entity	Attribute	Type	Description
Farmer	id, nom, contact	integer, text	Operating owner.
Farm	id, nom, fermier	integer, text, reference	Operation attached to a farmer.
Site	id, nom	integer, text	Beekeeping location.
Site	latitude, longitude	decimal (degrees)	Geolocation.
Site	altitude	decimal (configurable unit)	Site altitude.
Site	dateMiseEnOeuvre, dateDemenagement, dateCloture	date	Site life cycle, the last two optional.
Hive	id, modele	integer, text	Hive hosted by a site.
Hive	nbHausses	integer (0 to 5)	Number of extension levels.
Hive	ferme	reference	Owning farm (distinct from the location site).
Hive	etat	enumeration	Life-cycle state (created, populated, active, splitting, harvesting, closed), see figure 7.8.
Hive	agentResponsable	reference	Agent responsible at the given time.

Entity	Attribute	Type	Description
Compartment	id, type	integer, enumeration	Hive compartment: body or super.
Compartment	capaciteCadres	integer (1 to 10)	Number of frame slots.
Frame	id, slot, type	integer, enumeration	Frame installed on an identified slot.
Frame	poids	decimal (configurable unit)	Frame weight, basis of the honey-quantity calculation.
Agent	id, nom, role	integer, text, enumeration	Beekeeper, supervisor, manager or administrator.
Schedule	id, ruche, agent	integer, reference	Visit schedule.
Schedule	datePrevue, statutApprobation	date, enumeration	Planned date and supervisor approval.
Schedule	superviseur	reference	Supervising agent who approves the schedule.
Visit	id, ruche, agent	integer, reference	Visit carried out.
Visit	planning	reference	Approved schedule behind the visit (optional).
Visit	date, heure, duree	date, time, integer (min)	Timestamp and duration.
Visit	raison	enumeration	Reason for the visit.
Visit	constatations, actionsPrevues, actionsEffectuees, recommandations	text	Report content.
Visit	effectifQualitatif, etatSante	enumeration	Swarm assessment.
Visit	productivite	integer (1 to 3)	Productivity level.
Photo	id, url, visite	integer, text, reference	Photo attached to a visit report.
Measurement	id, ruche, typeIndicateur	integer, reference, enumeration	Measurement of an indicator.
Measurement	valeur, unite	decimal, text	Value and configurable unit.
Measurement	horodatage, latitude, longitude	datetime, decimal	Time-stamped and geolocated measurement.
Harvest	id, ruche, date	integer, reference, date	Honey harvest.
Harvest	quantiteMiel, lotQR	decimal (configurable unit), text	Quantity and batch identifier.

Entity	Attribute	Type	Description
Threshold	id, type, valeur, unite	integer, enumeration, decimal, text	Configurable system threshold.

Reference schema: the dictionary above is the business view of the data. Its normalised relational translation (keys, SQL types and `CHECK` constraints) constitutes the **logical data model** (LDM), which is authoritative for the implementation and is presented in appendix G (figure G.1). The name correspondence between the dictionary, the class diagram and the LDM is given by table G.1.

5.3 Results and calculation rules (outputs)

- **Honey quantity of a hive**: sum of the weights of the honey frames of the supers, minus the tare of the elements.

$$\text{HoneyQuantity}(h) = \sum_{f \in \text{honeyFrames}(h)} \text{weight}(f) - \text{tare}(h)$$

This value is exposed by the web service via `getZummHoneyActualQuantity(zummID)`.

- **Return on investment** of a site or a hive:

$$\text{ROI} = \frac{\text{Valued production}}{\text{Operation costs}}$$

- **Harvest alert**: triggered if production exceeds the configurable threshold.

$$\text{Production}(h) > \text{ProductionThreshold} \Rightarrow \text{harvest or extension alert}$$

- **Drop alert**: triggered if the relative drop exceeds the threshold.

$$\frac{V_{\text{prev}} - V_{\text{current}}}{V_{\text{prev}}} > \text{DropThreshold} \Rightarrow \text{inspection alert}$$

- **Unit conversion** to the reference unit, to compare heterogeneous measurements:

$$V_{\text{reference}} = V_{\text{measurement}} \times \text{conversionFactor}(\text{unit})$$

5.4 Typical queries

The results rely on queries executed via JDBC, for example for the honey quantity of a hive: `SELECT SUM(quantiteMiel) FROM recolte WHERE ruche = zummID`.

CHAPTER 6

Detailed use cases

This part describes how the system carries out the users' operations. The actors are the farmer, the beekeeping agent, the supervising agent, the manager, the administrator and the third-party applications.

The overall use-case diagram (figure 6.1) presents all the actors, the generalisation of the supervising agent from the beekeeping agent, as well as all the dependencies between cases (include and extend relationships).

6.1 UC1: Plan and approve a visit

- **Actor**: supervising agent.
- **Preconditions**: the hive exists and the agent is authenticated.
- **Main scenario**: the agent proposes a visit date, the supervisor reviews the schedule then approves, the system records the approved status.
- **Alternatives**: the supervisor refuses and requests a new date.
- **Postcondition**: the visit is scheduled and visible in the calendar.

6.2 UC2: Carry out a visit and fill in the report

- **Actor**: beekeeping agent responsible for the hive.
- **Preconditions**: a visit is planned and approved.
- **Main scenario**: the agent enters date, time, duration, reason, findings, planned and carried-out actions, recommendations, then assesses the population, the health status and the productivity from 1 to 3.
- **Alternatives**: offline entry then deferred synchronisation, adding photos.
- **Postcondition**: the report is saved and the calendar cell switches to the carried-out status.

6.3 UC3: View the production dashboard

- **Actor**: farmer or owner.
- **Preconditions**: production measurements exist.
- **Main scenario**: the farmer filters by farm, site and period, the system displays the hives whose production exceeds the threshold and proposes harvest or extension.

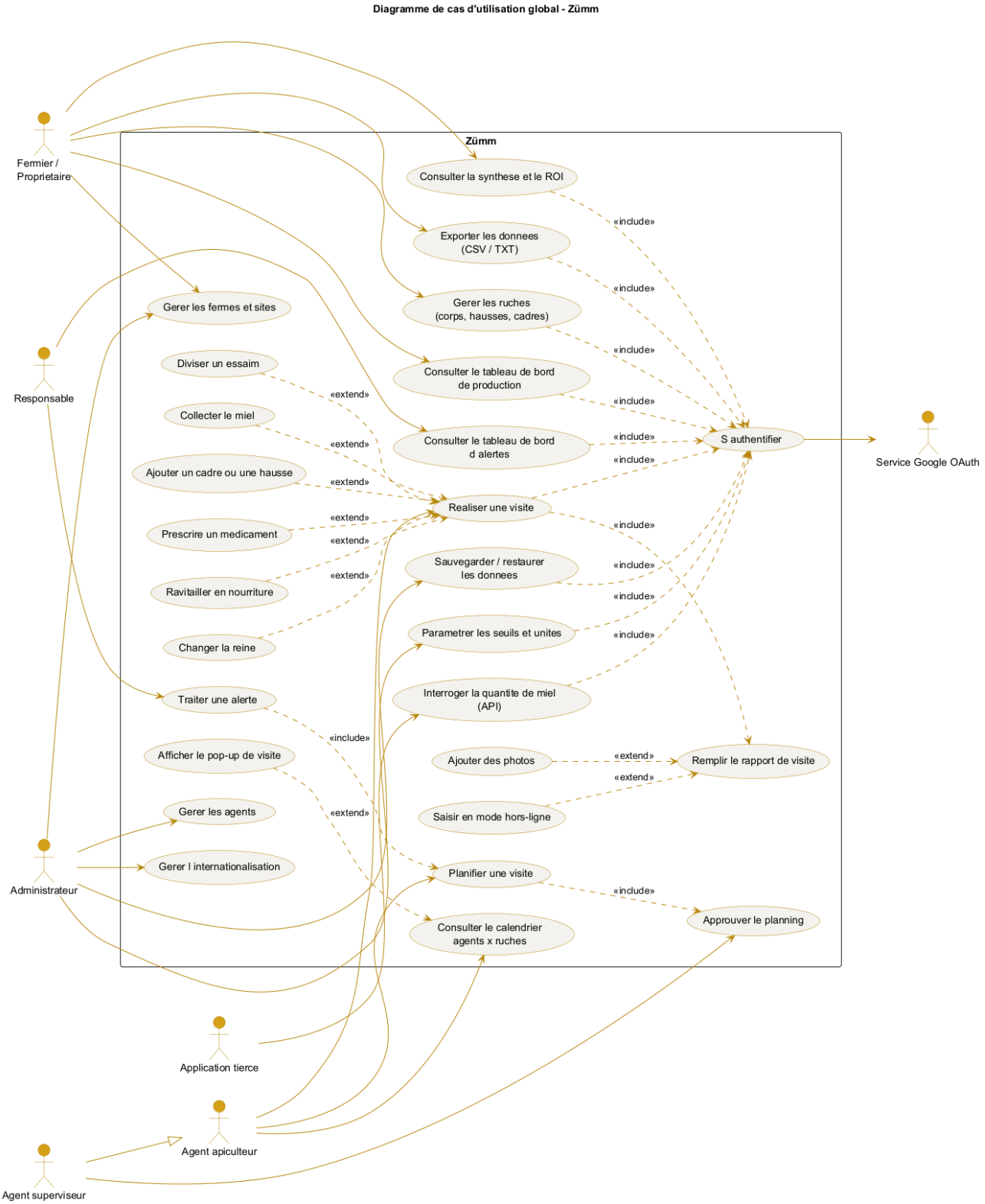

Figure 6.1. Overall use-case diagram with actors and dependencies (include and extend).

- **Postcondition**: the farmer decides on an operation.

6.4 UC4: Handle a health alert

- **Actor**: manager.
- **Preconditions**: an abnormal drop has been detected.
- **Main scenario**: the system flags the hive concerned, the manager plans an imminent inspection.
- **Postcondition**: an inspection visit is scheduled.

6.5 UC5: Query the honey quantity via the API

- **Actor**: third-party application (Excel or other).
- **Preconditions**: the application has secure access to the web service.
- **Main scenario**: the application calls `getZummHoneyActualQuantity(zummID)`, the service computes and returns the honey quantity.
- **Postcondition**: the value is returned in the XML web-service format.

CHAPTER 7

Design (UML models)

The design file comprises the following UML diagrams. Each one is described by its expected content in order to guide the modelling.

Diagram	Expected content
Use case	Actors (farmer, agent, supervisor, manager, administrator, third-party application) and main cases (see figure 6.1).
Class	Entities of the data dictionary and their relationships (Farmer 1 to n Farm, Farm 1 to n Site, Site 1 to n Hive, Hive 1 to n Body or super, Body or super 1 to n Frame) and service methods.
Object	Snapshot of a site containing three populated hives with their frames.
Sequence	Course of carrying out a visit (Agent, JSP UI, Servlet, EJB, JDBC, database).
Activity	Process of planning, approving, executing and closing a visit.
State machine	Life cycle of a hive (created, populated, active, splitting, harvesting, closed).
Interaction or collaboration	Same scenario as the sequence, in the form of a collaboration between objects.
Package	Presentation, control, business, data-access, internationalisation, configuration and web-services layers.
Component	Application components (web application, OAuth service, API service, i18n module, configuration module, database).
Deployment	Distribution across the browser, the J2EE application server, the DBMS, the Google OAuth service and the sensors.

7.1 Class diagram

The class diagram, presented in horizontal alignment, describes the business entities, their attributes, the service method `getZummHoneyActualQuantity` and the relationships with their cardinalities.

7.2 Layered view (package)

7.3 Deployment view

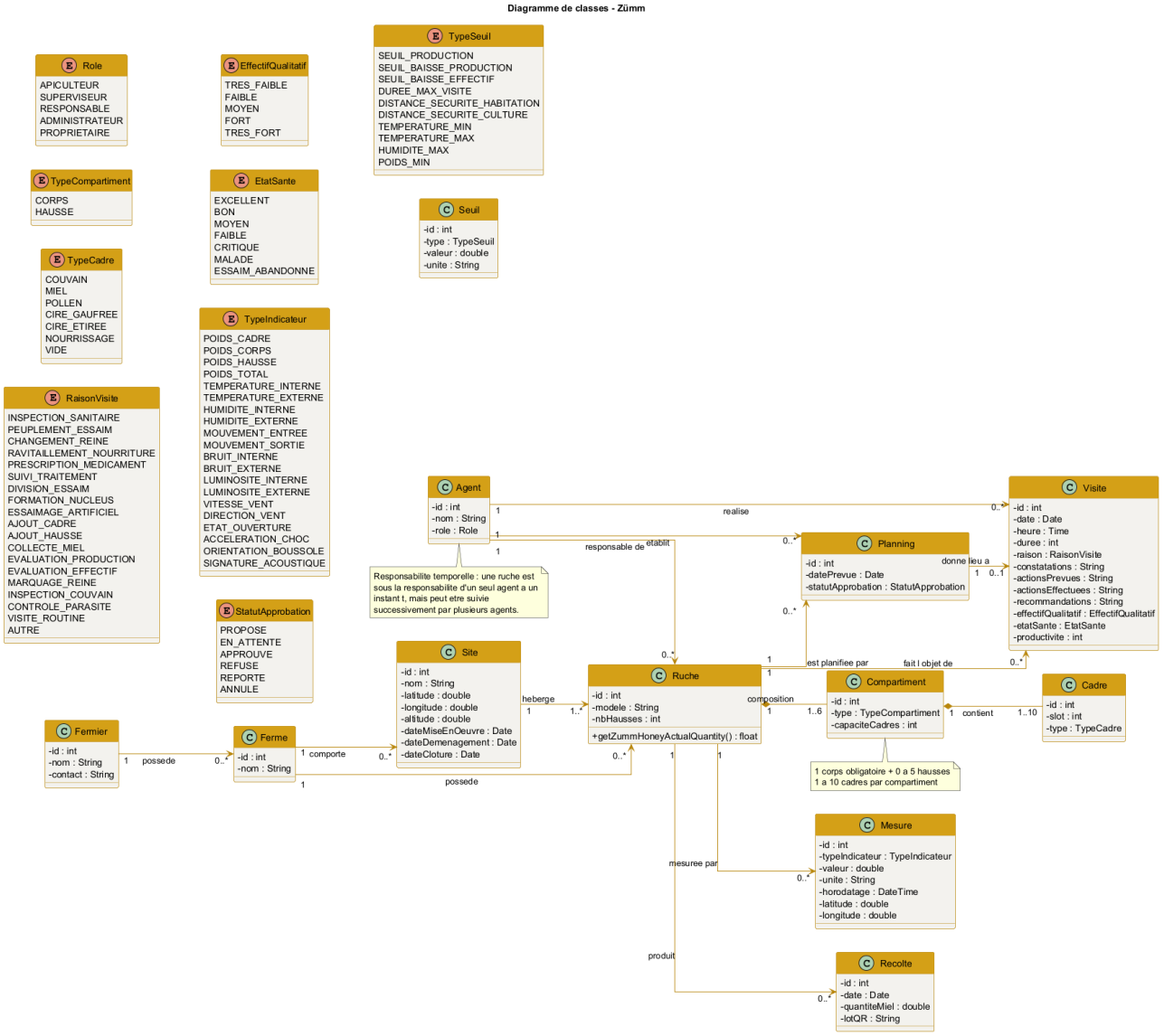

Figure 7.1. Class diagram of the Zümm system (horizontal alignment).

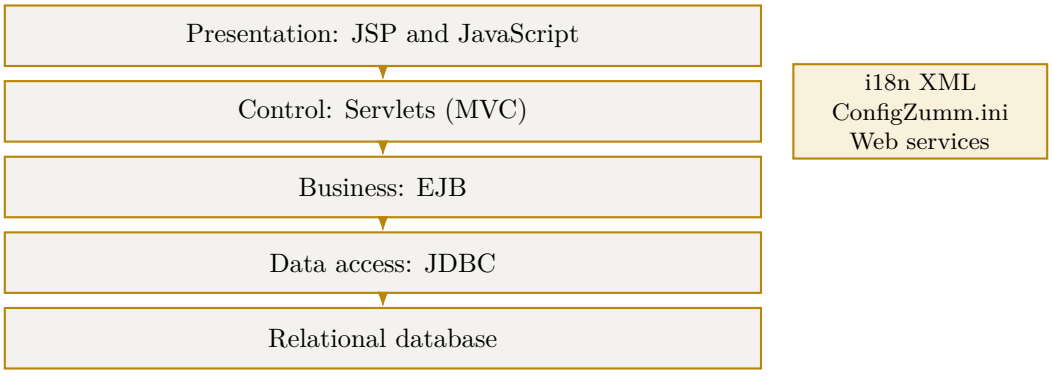

Figure 7.2. Multi-tier architecture in loosely coupled layers.

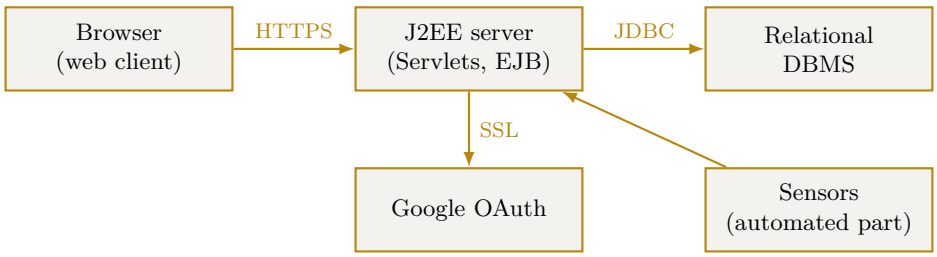

Figure 7.3. Deployment diagram of the system.

7.4 Sequence diagram (UC2)

The sequence diagram below details use case UC2, carry out a visit and fill in the report. It shows the passage through the layers (agent, UI, servlet, EJB, database) as well as the online and offline cases with deferred synchronisation.

7.5 Authentication sequence (Google OAuth)

The authentication flow follows the *Authorization Code* flow of OAuth 2.0. The browser never carries the client secret: the exchange of the authorization code for the access token is done **server to server**, over an SSL channel authenticated by an X.509 certificate. The application session is then opened by means of a secure cookie (`HttpOnly`, `Secure`), and the profile returned by Google is associated with an existing `Agent` (or creates one pending authorisation). In the demonstration, a local authentication fallback is provided in case the provider is unavailable.

7.6 Object diagram

The object diagram presents a snapshot of a site hosting three hives, with the detail of the composition of a hive into body, super and frames.

7.7 Activity diagram

The activity diagram models the process of planning, approving and carrying out a visit, with the online and offline variants.

7.8 State-machine diagram

The state-machine diagram describes the life cycle of a hive, from its creation to its closure.

7.9 Collaboration diagram

The collaboration diagram takes up the UC2 scenario in the form of numbered messages exchanged between the objects.

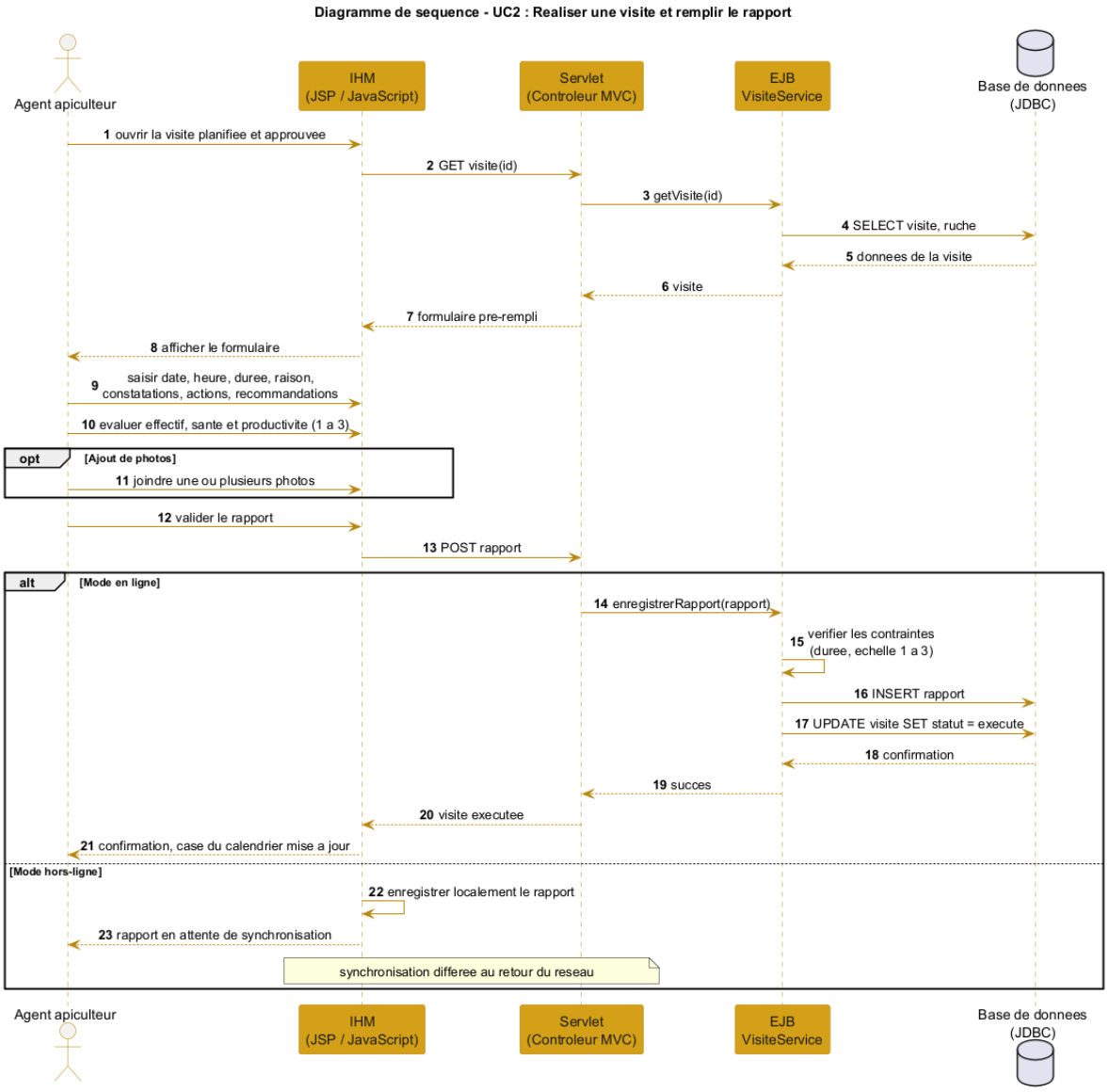

Figure 7.4. Sequence diagram of use case UC2 (carry out a visit).

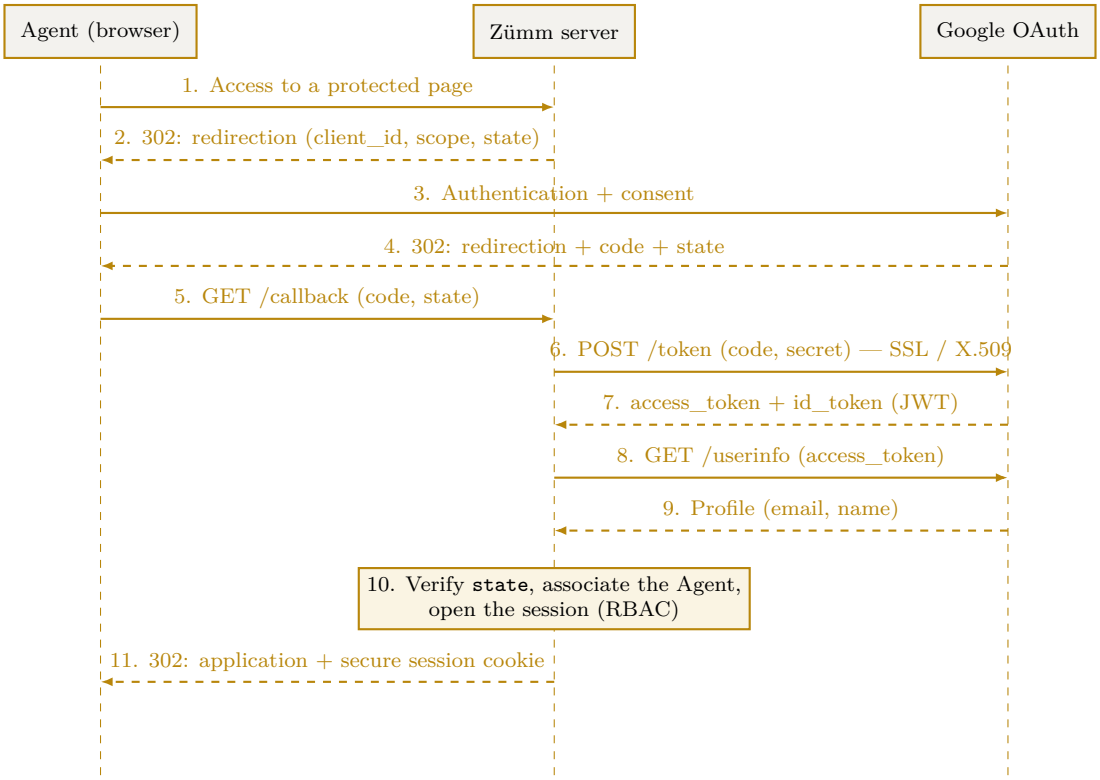

Figure 7.5. OAuth 2.0 authentication sequence (Authorization Code) with Google.

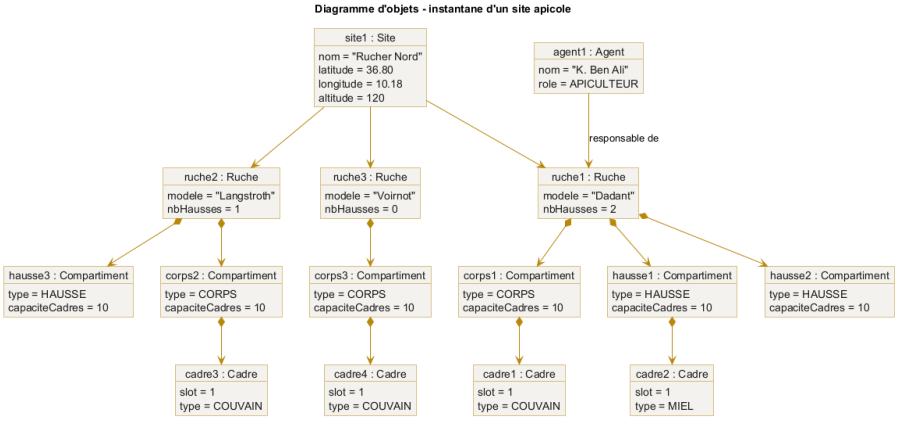

Figure 7.6. Object diagram, snapshot of an apiary site.

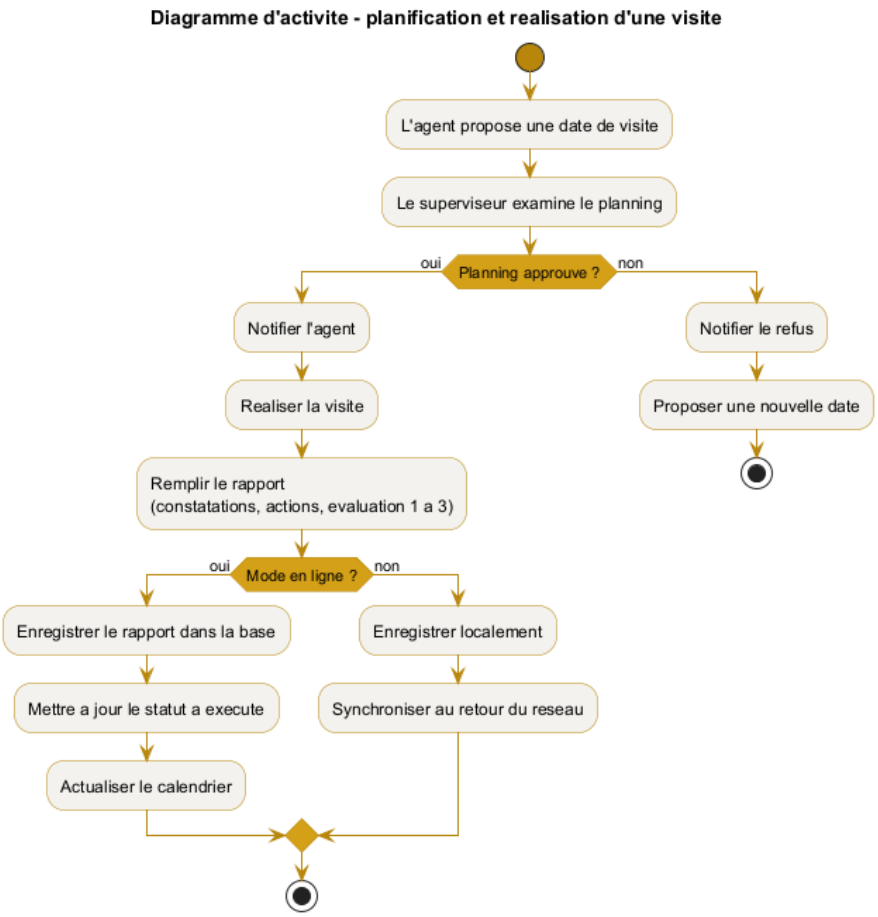

Figure 7.7. Activity diagram of the visit process.

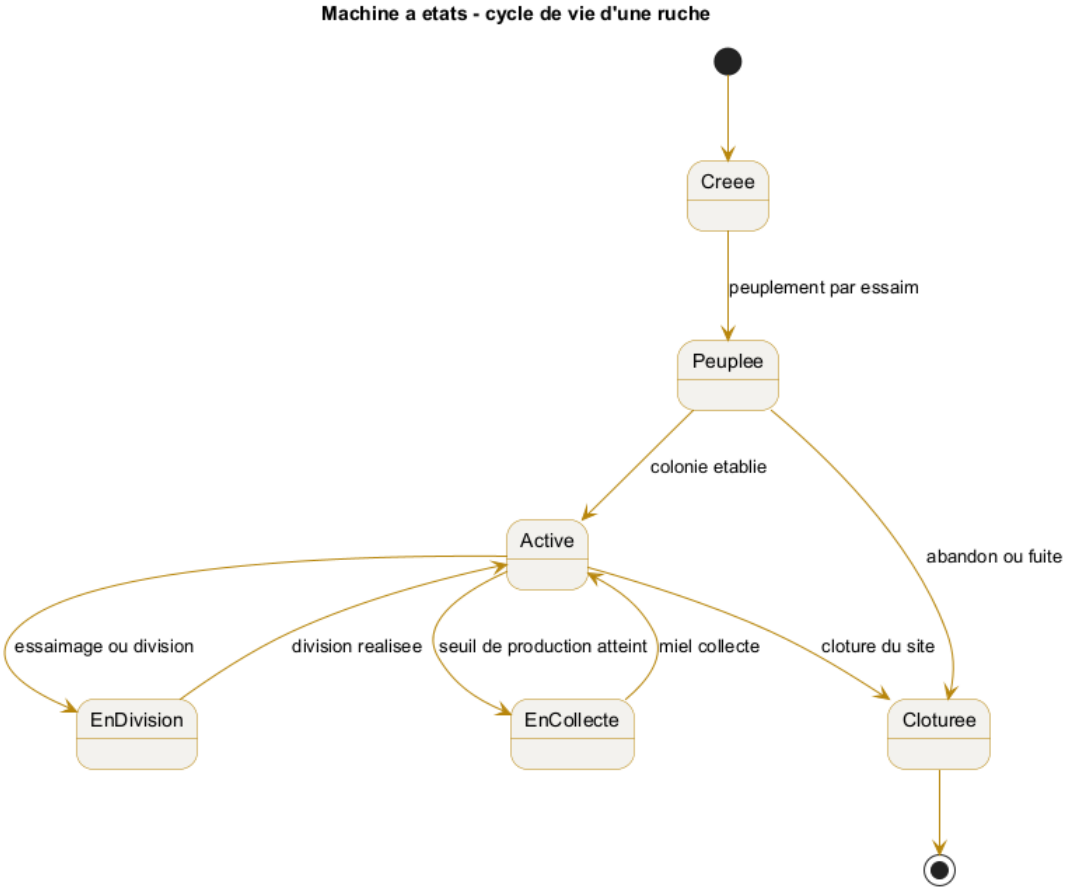

Figure 7.8. State machine of the life cycle of a hive.

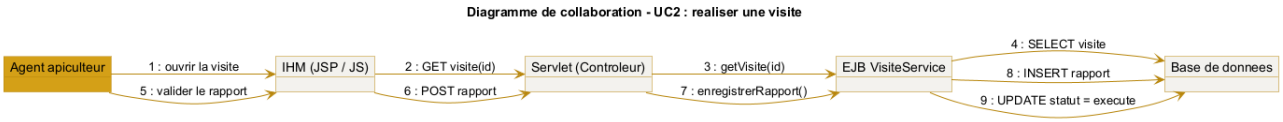

Figure 7.9. Collaboration diagram of use case UC2.

7.10 Component diagram

The component diagram shows the organisation of the application components, their interfaces and their dependencies, including the service interface exposing the honey quantity.

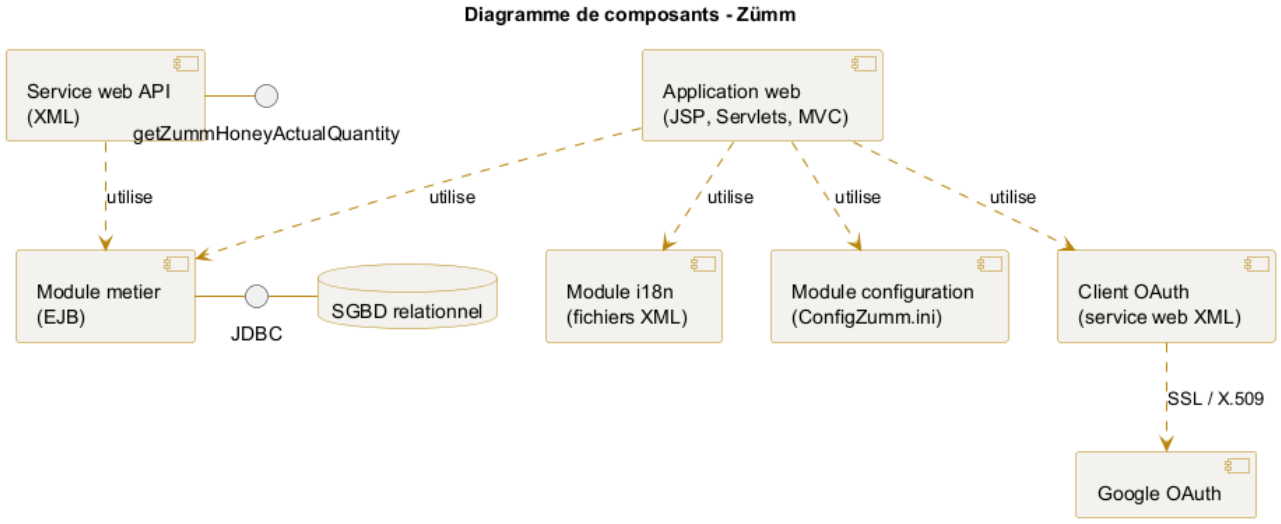

Figure 7.10. Component diagram of the Zümm system.

CHAPTER 8

Constraints and technical requirements

8.1 Architecture

A **scalable, evolvable, flexible, multi-tier and loosely coupled** architecture, relying on **J2EE** as the main infrastructure.

8.2 Imposed technical requirements

Requirement	Description
MVC	Model / view / controller separation.
Servlet	Server-side request processing.
JSP	Presentation front-end.
EJB	Business components.
JDBC	Data access.
Internationalisation	UI texts externalised in an XML file, open to offline translation with no language limit, FR, EN and AR at minimum in the demonstration.
Configuration	Parameterisation via the text file `ConfigZumm.ini`.
Inter-component security	**X.509** certificates and **SSL** protocol.
User authentication	**Google OAuth** via an XML web service and its API.
Third-party API	XML web service exposing `float getZummHoneyActualQuantity(int zummID)` to retrieve the honey quantity of a hive.
Front-end	JavaScript and free frameworks, respecting the imposed usability.

8.3 Non-functional requirements

- **Usability, user-friendliness and simplicity** of use, interface consistency and quality of the graphic design (of paramount importance).
- **Scalability**: openness to adding languages, units and indicators.
- **Interoperability**: web services queryable by third-party applications.
- **Security**: encryption of exchanges and strong authentication.

- **Design quality**: compliance with the **SOLID** principles and use of **design patterns** to guarantee loose coupling and scalability, as detailed in appendix F.

Design principles adopted: the architecture applies the five **SOLID** principles (single responsibility, open/closed, Liskov substitution, interface segregation, dependency inversion) and a set of **design patterns** (Composite, State, Strategy, Observer, Command, Factory Method, Builder, Adapter, Facade, Singleton, Proxy). Their application to the system, justified "before / after", is detailed in appendix F.

8.4 Design questions to cover

The design must explicitly answer the following questions:

1. What are the **inputs** (list, data dictionary and types) and **outputs** (list, dictionary, types, formulas and calculation queries) of the system?
2. What does the solution consist of?
3. Who are the typical users and their access rights to the features?
4. How does the system carry out the user operations?
5. How is the system packaged and deployed?
6. How do the elements interact and in what order / chronology?
7. Why, how and where was each technology used?

8.5 Justification of the technology choices

The following table answers the why, the how and the where of each imposed technology. The **concrete implementations** of this foundation (application server, DBMS, front-end libraries, web-service style) and the **justified comparison** of the evaluated alternatives are detailed in appendix D.

Technology	Why	Where
MVC	Separate the responsibilities and ease maintenance	General organisation of the application
Servlet	Process the requests and orchestrate the processing	Control layer
JSP	Produce the dynamic pages	Presentation layer
EJB	Encapsulate the business logic and make it reusable	Business layer
JDBC	Access the relational database in a standard way	Data-access layer
XML i18n	Externalise the texts for offline translation	Internationalisation module
ConfigZumm.ini	Parameterise the system without recompilation	Configuration module
Google OAuth	Authenticate users without managing passwords	Access to the system
XML API web service	Expose the data to third-party applications	Interoperability interface

Technology	Why	Where
X.509 and SSL	Encrypt and authenticate the exchanges	Inter-component communications
JavaScript	Enrich the usability of the front-end	Presentation layer

8.6 Packaging and deployment

- **Packaging**: the application is packaged as a Java enterprise archive (WAR for the web part, EAR if the EJBs are grouped), including the internationalisation XML files and the `ConfigZumm.ini` file.
- **Server**: deployment on a J2EE application server providing a servlet container and an EJB container.
- **Database**: relational DBMS accessed via JDBC.
- **Security**: installation of the X.509 certificates and activation of SSL for the exchanges, configuration of Google OAuth access.
- **Configuration**: the thresholds, units and active modules are adjusted in `ConfigZumm.ini` without redeployment.

The physical distribution is illustrated by the deployment diagram (figure 7.3).

CHAPTER 9

Budget envelope and resources

9.1 Framework

The project falls within an **academic (mini-project)** framework. The budget envelope therefore mainly translates into the **human, software and hardware resources** mobilised, with no direct financial cost.

9.2 Mobilised resources

Resource	Detail
Human	Student project team (design, development, testing, documentation).
Software	J2EE application server, relational DBMS, IDE, UML tools, compilation chain.
Hardware	Development workstations, simulated sensors for the indicators part.
External	Google OAuth account, X.509 certificates (self-signed in the demonstration).

Major academic constraint: any use of code generators (ChatGPT, DeepSeek and derivatives) is prohibited and will be considered fraud in the examination.

CHAPTER 10

Schedule and deliverables

10.1 Expected deliverables

- Functional application (J2EE source code + front-end).
- Test plan and execution results.
- Design file (UML diagrams).
- Project report, poster and oral presentation.

10.2 Grading grid (scale)

Criterion	Points
Usability	4
Tested features (test plan + execution)	4
Technologies	**4**
MVC	0.5
Servlet	0.5
JSP front-end	0.25
XML file and internationalisation	0.5
Configuration text file	0.25
OAuth client web service	0.5
XML API web service	0.5
JDBC	0.5
X.509 / SSL	0.25
EJB	0.25
Design (UC, classes, objects, sequence, activity, state, interaction/collaboration, package, component, deployment)	4
Documentation	**4**
Report (structure, consistency, format)	2
Poster	1
Presentation	1
Total	**20**

10.3 UML diagrams to produce

Diagrams of: use case, classes, objects, sequence, activity, state machine, interaction / collaboration, package, component and deployment. The expected content of each is detailed in chapter 7 on the design.

10.4 Provisional schedule

The schedule is expressed in relative weeks, with the intermediate deliverables associated with each phase.

Phase	Weeks	Deliverable
Analysis and requirements specification	W1 to W2	Validated requirements specification
Design (UML)	W3 to W4	Design file
CRUD core development	W5 to W7	Model and entity management
Dashboards and alerts	W8 to W9	Calendar, alerts and summaries
Web services, security, i18n	W10 to W11	OAuth, API, X.509 and SSL, XML
Testing and validation	W12	Executed test plan
Documentation and presentation	W13	Report, poster and defence

CHAPTER 11

Test plan and validation

The test strategy covers the business functions, the management constraints and the technical requirements. Each test case specifies the target function, the preconditions, the steps and the expected result.

11.1 Test cases

ID	Function	Steps	Expected result
TC01	Hive CRUD	Create, read, modify, delete a hive	Compliant and persisted operations
TC02	Frame constraint	Add an 11th frame to a body	Rejected, maximum of 10 frames
TC03	Super constraint	Add a 6th super	Rejected, maximum of 5 supers
TC04	Base constraint	Remove the last frame of the body	Rejected, at least 1 frame
TC05	Visit schedule	Propose then approve a visit	Approved status recorded
TC06	Visit report	Fill in all fields and save	Report persisted, cell carried out
TC07	Production alert	Exceed the production threshold	Harvest alert displayed
TC08	Drop alert	Simulate a population drop	Inspection alert displayed
TC09	ROI calculation	Enter costs and production	ROI computed correctly
TC10	Honey API	Call getZummHoneyActualQuantity	Correct quantity returned
TC11	Internationalisation	Switch FR, EN then AR	Texts translated via the XML file
TC12	Configuration	Modify a threshold in ConfigZumm.ini	New threshold taken into account
TC13	Authentication	Log in via Google OAuth	Access granted
TC14	Security	Verify SSL and X.509 certificate	Encrypted and validated exchanges

ID	Function	Steps	Expected result
TC15	Export	Export the records	CSV and TXT files generated
TC16	Offline mode	Enter without a network then reconnect	Successful synchronisation

11.2 Traceability matrix

The matrix links each requirement to its use case, to the classes and tables that carry it, to the screen (mockup) concerned and to the test cases that validate it. It ensures end-to-end traceability, from the need to the verification.

Requirement	UC	Classes / Tables	Screen	Tests
Entity management and constraints	—	Hive, Compartment, Frame	Hive form (fig. G.5)	TC01–TC04
Planning and approval	UC1	Schedule, Agent	Calendar (fig. G.2)	TC05
Visit execution and report	UC2	Visit, Photo	Report (fig. G.4)	TC06
Production dashboard	UC3	Harvest, Measurement, Threshold	Prod. dashboard (fig. G.3)	TC07, TC09
Health / drop alert	UC4	Measurement, Threshold	Calendar	TC08
API web service (honey)	UC5	Hive, Harvest	—	TC10
Internationalisation	—	(i18n XML)	All	TC11
Threshold configuration	—	Threshold	Prod. dashboard	TC12
OAuth authentication	—	Agent	Login (fig. 7.5)	TC13
Exchange security	—	(SSL / X.509)	—	TC14
Portability and export	—	(CSV/TXT export)	All	TC15
Offline mode	UC2	Visit (local queue)	Report	TC16

CHAPTER 12

Beekeeping domain reference and good practices

This chapter specifies the domain knowledge that underpins the management rules, the configurable thresholds and the system indicators. It constitutes the functional basis on which the dashboards and the alerts rely.

12.1 Honey production factors

A colony's production depends directly on the weather, pollen availability and nectar flow. When the nectar flow is high, the queen lays more, the population peaks and the yield reaches its maximum. In a period of food shortage, laying and population decrease. The system must therefore correlate the measured production with periods and seasons, which justifies the filtering of the dashboards by month, week, season and year.

12.2 Location and configuration of a site

The choice of a site's location determines productivity. The following reference values can feed the fields and the input checks of the site-management module.

Criterion	Reference value
Proximity to resources	Nectar and pollen sources about 1 km away.
Access to water	Accessible drinking water, several litres consumed per day and per colony.
Distance from dwellings	100 m in a forest area, 200 m in a shrubland area, 300 m in an open area.
Distance from treated crops	At least 3 to 4 km from conventional crops using pesticides.
Elevation and inclination	Hives raised about 1.5 m off the ground and slightly tilted to drain moisture.
Protection	Shelter from intense sun, wind and floods.

Figure 12.1. Example of a hive installed on a site: elevation on legs and landing board.

12.3 Hive types and reference yield

Type of hive	Characteristics and yield
Traditional fixed-comb hive	6 to 10 kg of comb honey per season, not recommended in organic beekeeping.
Top-bar hive	Standard width of 32 to 35 cm, independent inspection of the combs.
Frame hive (Langstroth, East-African)	Honey combs reusable for several seasons after harvest.

These orders of magnitude serve as a basis for setting the production thresholds and estimating the return on investment per hive.

12.4 Classification of hive models

Two large families of hives are distinguished, whose modelling must remain open to cover local and international uses.

Family	Representative models
Frame hives (vertical)	Dadant, Langstroth, Voirnot, Layens, Aumônière, National (GB), WBC, Tonelli.
Traditional frameless hives	Warré, Kenyan TBH, Tanzanian TBH, log, straw (skep).

Family	Representative models
Other regional models	Lucitana, Oksman, Smith, CDB, Duvauchelle, Jarry, Congres, Zanders, Slovenian models.

In France, the most common models are the Dadant, Langstroth and Voirnot hives. All frame hives share a common composition, with dimensions precise to the millimetre, which reinforces the chosen data model.

Model anchor: the project's reference model is the Dadant hive, whose body holds up to 10 frames. This value grounds the maximum capacity of 10 frames per body or per super adopted in the management rules. The system nevertheless remains configurable to support other models.

12.4.1 Common composition of a frame hive

Element	Role
Floor / landing board	Base of the hive and landing zone for the bees.
Body (base)	Main brood compartment, holds the base frames.
Super	Extension level intended for honey harvesting.
Frame	Removable support of the wax combs, installed on an identified slot.
Queen excluder	Separates the body from the supers and confines the queen to the body.
Crown board and roof	Close and protect the hive from bad weather.
Feeder	Allows the food replenishment of the colony.

12.5 Colony inspection protocol

Inspections are preferably carried out on sunny days and focus on precise checkpoints that structure the visit report.

- brood development (eggs, larvae, capped cells)
- filling of the cells with honey and pollen
- presence of parasites or diseases
- collection of nectar, pollen and propolis

Operational constraint: a visit must not exceed 45 minutes. Beyond that, the colony's agitation increases and spreads to the neighbouring hives. The *duration* field of the visit report can be checked against this reference limit.

12.6 Swarming and colony splitting

An imminent swarming is signalled by the presence of queen cells on the edges of the combs. A few days before the emergence of a new queen, the old queen leaves the colony with half the workers. The

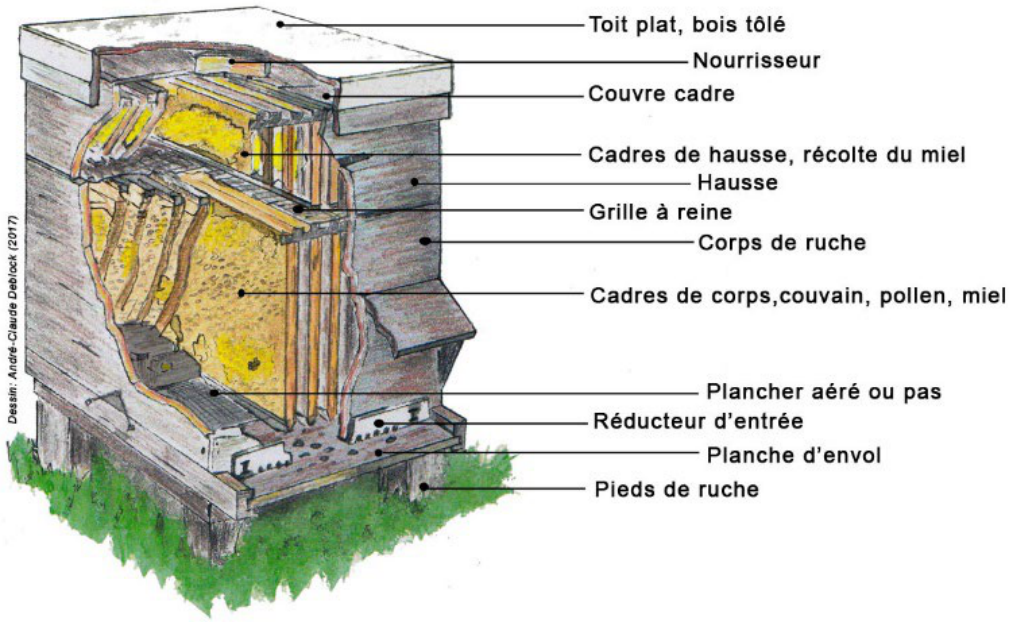

Figure 12.2. Cross-section of a frame hive and role of each element (roof, feeder, crown board, super, queen excluder, body, floor).

system must therefore make it possible to anticipate the split. Three reference methods are provided as operation reasons.

Method	Principle
Nucleus formation	Remove 1 to 2 brood combs with bees to a new hive.
Queen nucleus	Transfer the queen with a comb free of queen cells.
Artificial swarming	Move the mother hive and place a new hive at the old location with 2 to 3 combs of young brood.

12.7 Well-being indicators and causes of decline

A healthy colony presents a well-developed brood nest at the various stages, cells filled with resources, visible foraging activity and the absence of parasites or diseases. These criteria feed the qualitative assessment of the health status and the productivity level rated from 1 to 3 in the visit report.

The absconding or abandonment of a hive results from excessive disturbances or inadequate conditions (lack of food or water, extreme temperatures). Leaving a honey reserve to the colony during the harvest reduces this risk. These phenomena justify the alerts of sudden population or production drop and the associated configurable thresholds.

12.8 Impact on the configurable thresholds

System parameter	Business anchor
Honey-harvest threshold	Reference yield per hive type and per season.
Drop-alert threshold	Abnormal drop linked to absconding, disease or food shortage.
Maximum visit duration	Limit of 45 minutes per inspection.
Site safety distances	100, 200 or 300 m depending on the area, 3 to 4 km from treated crops.
Analysis periods	Correlation of production and season (nectar flow).

APPENDIX A

Appendix: Physical structure of a hive

For the record, a movable-frame hive is made up, from bottom to top, of the following elements: floor / landing board, body (base) holding the brood frames, queen excluder, one or more supers holding the honey-harvest frames, crown board and roof. This structure justifies the composition rules (mandatory body, up to 5 supers, 1 to 10 frames per element) adopted in the business model.

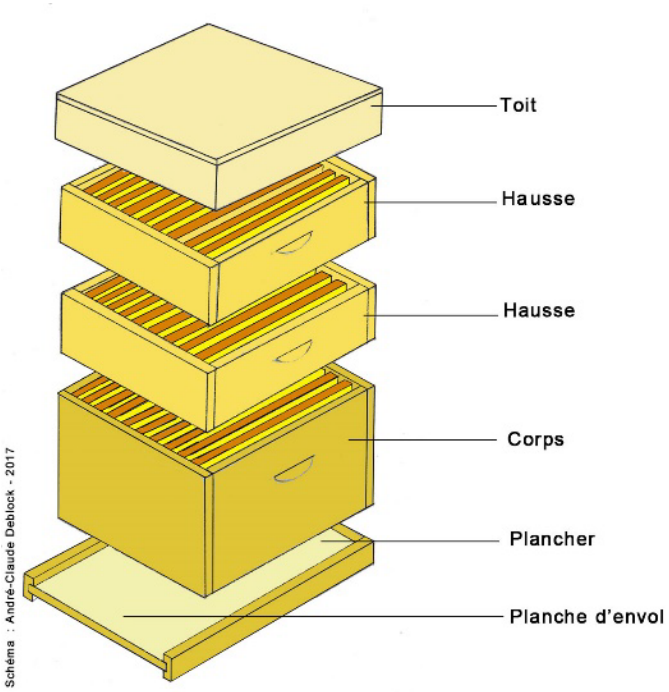

Figure A.1. Exploded view of a frame hive: roof, supers, body, floor and landing board.

APPENDIX B

Glossary

Term	Definition
Base or body	Base compartment of the hive holding the brood frames.
Super	Extension level added to the body for honey harvesting.
Frame	Removable support of the wax combs, installed on a slot.
Slot	Identified location of a frame in a body or a super.
Swarm	Colony of bees populating a hive.
Swarming	Departure of part of the colony with the queen to found a new colony.
Nucleus	Small colony formed to split a swarm or raise a queen.
Queen	The colony's single reproductive female.
Brood	All the eggs, larvae and pupae of the colony.
Foraging	Activity of collecting nectar and pollen by the bees.
Propolis	Resinous substance collected by the bees.
Queen excluder	Grid that confines the queen to the body and separates the supers.
Feeder	Device for the food replenishment of the colony.
Configurable threshold	Configurable reference value triggering an alert.
ROI	Return on investment, ratio between valued production and costs.
CRUD	Create, read, update and delete operations.
i18n	Internationalisation, adaptation to languages and unit systems.
EJB	Business component of the J2EE infrastructure.
Servlet	Server component processing web requests.
JSP	Server page producing the dynamic display.
JDBC	Access interface to relational databases.
OAuth	Authorization protocol allowing delegated authentication.
X.509 and SSL	Certificates and protocol ensuring encrypted and authenticated exchanges.
TLS	Encryption protocol for network exchanges (transport layer), basis of HTTPS.
REST / JSON	Web-API style and text format for exchanging data (measurement ingestion).
MQTT	Lightweight messaging protocol (publish/subscribe) suited to sensors and constrained networks.

Term	Definition
JWT (token)	Signed identity-bearing token, issued by the OAuth provider.
RBAC	Role-based access control, applied server-side (see appendix G).
GDPR	General Data Protection Regulation. Governs personal data.
Retention	Duration for which a datum is kept before archiving, anonymisation or purge.
Encryption at rest	Encryption of stored data (sensitive columns) in the database.
Pseudonymisation	Dissociation of a person's identity from their data, reversible under control.
Idempotency	Property of an operation that can be replayed without creating a duplicate (measurement ingestion).
Hysteresis	Persistence of a breach over several measurements before raising an alert (debounce).

APPENDIX C

References and sources

This requirements specification draws on the following sources.

Elaboration method

- Manager-GO, *Élaborer un cahier des charges* (Drawing up a requirements specification).
  https://www.manager-go.com/gestion-de-projet/dossiers-methodes/elaborer-un-cdc

Software design principles

- Alex so yes, *The SOLID principles*.
  https://alexsoyes.com/solid/
- Refactoring Guru, *Design patterns catalogue*.
  https://refactoring.guru

Beekeeping domain reference

- Organic Africa, *Improving honey production*.
  https://www.organic-africa.net/fr/agriculture-biologique/agriculture-biologique/animaux/apiculture/amelioration-de-la-production-de-miel.html
- Au Bon Miel, *The hives*.
  https://www.aubonmiel.com/les-ruches/

Features of market solutions

- HiveTracks, beekeeping management application.
  https://www.hivetracks.com/
- Apiary Book, apiary management solution.
  https://www.apiarybook.com/
- BeePlus, beekeeping manager (App Store).

Connected monitoring (precision beekeeping)

- Arnia, acoustic monitoring and weighing of hives.
  https://www.arnia.co/

- BroodMinder, temperature, humidity and weight sensors.
  https://broodminder.com/
- BeeHero, IoT platform and artificial-intelligence analysis.
  https://www.beehero.io/

User feedback and limits of the solutions

- Beekeeping Diary, *Best Beekeeping Apps 2026 (comparison)*.
  https://beekeeping-diary.eu/blog/en/best-beekeeping-apps-2026/
- HiveBook, *Free HiveTracks Alternatives*.
  https://hivebook.app/blog/hivetracks-alternatives-free
- Beesource Forums, *Problems with Hive Tracks*.
  https://www.beesource.com/threads/problems-with-hive-tracks.249656/
- ApiTech World, *Smart Hive Monitoring, a Beekeeper's Reality Check (2026)*.
  https://apitechworld.com/smart-hive-monitoring-a-beekeepers-reality-check-2026-review/

APPENDIX D

Appendix: Technology stack and comparison of alternatives

The technical foundation of **Zümm** (MVC, Servlet, JSP, EJB, JDBC, XML i18n, Google OAuth, X.509 / SSL, XML web service) is **imposed** by the requirements specification and by the grading grid. The freedom of choice therefore concerns the **concrete implementations** of this foundation: application server, DBMS, front-end libraries, web-service style. This appendix lists the technologies chosen layer by layer, then justifies each choice against its alternatives.

D.1 Chosen technology stack

The table below specifies, for each layer of the multi-tier architecture (figure 7.2), the concrete implementation chosen and its reference version. All the building blocks are **free and open source**, in line with the requirement of self-deployment without a subscription.

Layer / need	Chosen technology	Role in Zümm
Platform	Jakarta EE 10 on JDK 17 LTS	Current "J2EE" foundation: Servlet, JSP, EJB, JAX-WS
Application server	WildFly (Web + EJB container)	Runs the WAR/EAR archive, provides the containers
Control (MVC)	Jakarta Servlets + JSP + JSTL	Request routing, control layer
Business	Session EJB (stateless)	Management rules: ROI, thresholds, honey quantity
Data access	JDBC + HikariCP pool	SQL queries, connection management
DBMS	PostgreSQL (+ PostGIS)	Relational persistence, site geolocation
Presentation	HTML5 + JavaScript + Bootstrap 5	Responsive UI, web / mobile parity
Charts	Chart.js	Production, alerts and summary dashboards
Mapping	Leaflet + OpenStreetMap	Site map and foraging radii
i18n	XML files + `ResourceBundle`	FR / EN / AR texts, Arabic RTL support
Configuration	`ConfigZumm.ini` via Singleton	Thresholds, units and modules without redeployment

Layer / need	Chosen technology	Role in Zümm
Third-party web service	JAX-WS (SOAP / XML)	`getZummHoneyActualQuantity(int)`
Authentication	Google OAuth 2.0	Delegated login without password management
Exchange security	TLS / SSL + X.509 certificates	Encryption and inter-component authentication
Offline	PWA (Service Worker + IndexedDB)	Field entry without a network, deferred synchronisation
Build & tests	Maven, JUnit 5, Mockito, Arquillian	Compilation, unit and in-container tests

Note: moving from the historical name "J2EE" to **Jakarta EE 10** changes neither the concepts nor the APIs noted (Servlet, JSP, EJB, JDBC). It merely adopts the maintained version of the platform. The modernisation trajectory towards Spring Boot is treated separately in appendix E.

D.2 Justified comparison of the alternatives

For each open decision, the credible options were evaluated. The table keeps the option, cites the discarded alternatives and gives the justification with regard to the project requirements (self-deployment, free of charge, usability, geolocation, interoperability).

D.2.1 Application server

Option	Discarded alternatives	Justification of the choice
WildFly	GlassFish, Payara, Open Liberty, Apache TomEE	Complete and free **EJB + Servlet** container, fast startup, abundant documentation. TomEE remains lighter but less integrated. Payara also fits (production-oriented GlassFish fork).

D.2.2 Database management system

Option	Discarded alternatives	Justification of the choice
PostgreSQL	MySQL / MariaDB, H2, Oracle XE	Native **geolocation** (PostGIS) for the sites and foraging radii, good support of `CHECK` constraints (frame/super rules), time series of measurements. Free and without proprietary lock-in, unlike Oracle.

D.2.3 Third-party web-service style

Option	Discarded alternatives	Justification of the choice
JAX-WS (SOAP / XML)	JAX-RS (REST/JSON), gRPC	The requirements specification imposes an **XML web service** exposing `float getZummHoneyActualQuantity(int zummID)`. SOAP is its literal translation, with a WSDL contract queryable from Excel. A REST/JSON endpoint may complete the offer during modernisation (appendix E).

D.2.4 Mapping and foraging radii

Option	Discarded alternatives	Justification of the choice
Leaflet + OSM	Google Maps JS API, Mapbox	**Free and without a paid key** or billed quota, consistent with the self-deployment requirement. Simple drawing of the foraging circles (1, 2, 3 km) from the site coordinates.

D.2.5 Charts library

Option	Discarded alternatives	Justification of the choice
Chart.js	D3.js, ApexCharts, Highcharts	Low learning curve and direct rendering of the histograms and curves of the dashboards (production, ROI). D3 is more powerful but oversized. Highcharts is not free for commercial use.

D.2.6 Offline and mobile access

Option	Discarded alternatives	Justification of the choice
PWA (Service Worker + IndexedDB)	Native application (Android/iOS), Flutter, React Native	**Web / mobile parity** with a single codebase, field entry without a network then deferred synchronisation. Avoids the development and distribution of native applications, out of the academic scope.

Consistency with the academic constraint: all the chosen building blocks are free, documented and *justifiable* "why / how / where" (*Technologies* criterion of the grading grid, schedule chapter). None relies on a code generator: they are implemented by hand by the team, in compliance with the exam constraint.

APPENDIX E

Appendix – Technology choice: academic and market vision

This appendix answers the question of the *why* and the *how* of the development language. It confronts the **J2EE** foundation imposed by this requirements specification with the **Spring Boot** ecosystem, which has become the standard of enterprise Java development, and documents a modernisation trajectory without a change of language.

E.1 Positioning

The **Züimm** system is implemented in **J2EE (Java EE)** in accordance with the technical requirements (chapter 7 and grading grid): MVC, Servlets, JSP, EJB, JDBC, externalised internationalisation, secure web services (OAuth, X.509 / SSL) and an API queryable by third-party applications. This foundation embodies the core skills of the DSI programme: mastery of the layers of an information system, separation of responsibilities and interoperability.

Market observation: the historical Java EE building blocks (JSP, "bare" Servlets and EJB) are today considered *legacy*. Recruitment for new information systems very largely favours the **Spring Boot** ecosystem. This appendix keeps the **Java** language and demonstrates, technology by technology, the modern equivalent of each requirement.

E.2 Correspondence table: technical core

The following table sets each imposed (and graded) technology against its Spring Boot equivalent, as well as the concrete nature of the change.

J2EE (imposed)	**Spring Boot**	**What concretely changes**
MVC (manual)	Spring MVC (`@Controller`, `@RequestMapping`)	The pattern remains, routing becomes declarative through annotations instead of `web.xml`.

J2EE (imposed)	Spring Boot	What concretely changes
Servlet (`HttpServlet`, `doGet/doPost`)	`@RestController` / `@Controller`	No more `extends HttpServlet`: one annotated method = one endpoint. The `DispatcherServlet` orchestrates in the background.
JSP (presentation)	Thymeleaf (server-side) or React / Angular (SPA)	Modern templating without Java scriptlets. For a strong market effect, REST API + decoupled front-end.
EJB (`@Stateless`, EJB container)	Spring `@Service` / `@Component`	Same role (business logic, injection, transactions) without a heavy EJB container. `@Transactional` manages the transactions.
JDBC (manual SQL, `ResultSet`)	Spring Data JPA / Hibernate	Object-relational mapping replaces hand-written SQL. `JpaRepository` generates the CRUD (`JdbcTemplate` remains available).
i18n by XML file	`messages_fr/en/ar.properties` + `MessageSource`	Same principle of text externalisation, standard `.properties` format, still FR / EN / AR.
`ConfigZumm.ini`	`application.yml` + `@ConfigurationProperties`	Parameterisation without recompilation, injectable per profile. The thresholds and units become typed beans.
Google OAuth (home-made XML WS)	Spring Security OAuth2 Client	Google login in a few lines of configuration instead of a hand-written OAuth client, and more secure.
XML API web service `getZumm` `HoneyActual` `Quantity`	REST JSON API (`@GetMapping`) + OpenAPI / Swagger	JSON instead of XML, auto-generated documentation. XML remains possible via `produces = APPLICATION_XML`.
X.509 / SSL (manual)	Spring Boot SSL (`server.ssl.*`) + Keystore	Declarative HTTPS activation, same X.509 certificates, trivial configuration.
JavaScript (free front-end)	React / Vue / Angular (or Thymeleaf + JS)	Decoupled front-end consuming the REST API, simplified web and mobile parity.

E.3 Correspondence table: infrastructure and deployment

J2EE	Spring Boot	Benefit
Heavy application server (WildFly, GlassFish, WebLogic)	Embedded Tomcat / Jetty	The application starts on its own: `java -jar app.jar`, no more server to install.

J2EE	Spring Boot	Benefit
WAR / EAR packaging	Executable JAR (fat-jar)	A single self-contained artefact, ideal for the cloud and Docker.
Manual deployment on a container	Docker + `docker-compose`	Alignment with market DevOps, facilitated continuous integration.
Relational DBMS via JDBC	PostgreSQL / MySQL via JPA + Flyway	Versioned schema, reproducible across environments.
Manual dependency management (Ant)	Maven / Gradle + Spring Boot Starters	One dependency = one starter (`-web`, `-security`, `-data-jpa`).

E.4 Layer correspondence

The loosely coupled layered view required in chapter 7 (figure 7.2) is **fully preserved**: no layer disappears, only the implementation is modernised.

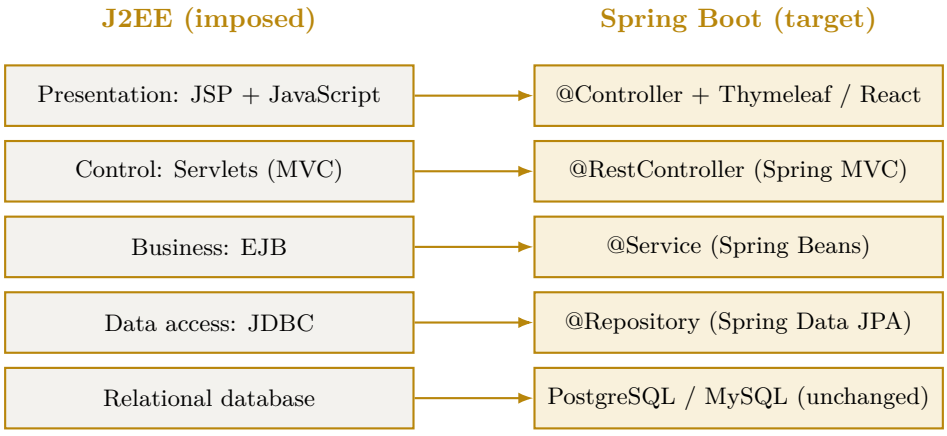

Figure E.1. Correspondence of the layers from J2EE to Spring Boot: preserved architecture, modernised implementation.

The cross-cutting modules follow the same logic: XML i18n → `MessageSource`, `ConfigZumm.ini` → `application.yml`, XML web service → REST API + OpenAPI.

E.5 Rationale: academic compliance and market vision

Zümm relies, in accordance with the requirements specification, on a **multi-tier, scalable and loosely coupled J2EE** architecture: MVC, Servlets, JSP, EJB, JDBC, internationalisation by an externalised file, secure web services (OAuth, X.509 / SSL) and an API queryable by third-party applications. This foundation was chosen and implemented **as required**, because it constitutes the project's evaluation reference and embodies the core skills of the DSI programme: mastery of the layers of an information system, separation of responsibilities and interoperability.

A **market-oriented** analysis nevertheless leads to an important observation. The historical Java EE building blocks (JSP, "bare" Servlets and EJB) are today considered *legacy*. Recruitment for new information systems very largely favours the **Spring Boot** ecosystem, which has become the *de facto* standard of enterprise Java development. Ignoring this reality would amount to delivering a technically compliant solution that is nevertheless **out of step with current professional practice**.

We have therefore adopted a **two-level reading approach**, without ever leaving the **Java** language, which guarantees the continuity of the acquired skills:

1. **Compliance level**: the deliverable fully respects the imposed technologies and the grading grid. The design diagrams (Agent → JSP → Servlet → EJB → JDBC sequence, layers, deployment) faithfully reflect this architecture.
2. **Modernisation level**: for each imposed technology, its Spring Boot equivalent is demonstrated (Servlet → `@RestController`, EJB → `@Service`, JDBC → Spring Data JPA, XML i18n → `MessageSource`, `ConfigZumm.ini` → `application.yml`, XML WS → REST API + OpenAPI, X.509 / SSL → declarative SSL configuration). This correspondence proves that **the target architecture remains identical**: only the implementation gains in concision, security and maintainability.

This choice presents a third advantage, in direct line with the outlets of the DSI programme. The requirements specification provides for a **sensor and predictive-analysis part** (swarming detection, acoustic processing). This module naturally lends itself to a **Python microservice** (pandas, scikit-learn) exposed to the Java back-end via REST, concretely illustrating the profession of **data analyst** cited in the study plan and the openness of a **loosely coupled architecture** to technological heterogeneity.

Conclusion: the project fully satisfies the academic requirements (J2EE) while documenting its **modernisation trajectory** (Spring Boot and data microservice). This dual reading turns a pedagogical constraint into a professional asset: it demonstrates not only the mastery of the fundamentals of the information system, but also the ability to evolve an architecture towards the standards sought on the 2026 job market.

APPENDIX F

Appendix – SOLID principles and design patterns

This appendix answers the question of the *how* of software quality. It documents the application of the five **SOLID** principles and a set of **design patterns** to the **Zümm** system, setting each decision against the initial design (*before*) and the refactored design (*after*). The objective is a **multi-tier, scalable and loosely coupled** architecture, compliant with the requirements of chapter 7.

Methodological sources: the definitions adopted follow the presentation of the SOLID principles from *alexsoyes.com* and the pattern catalogue from *refactoring.guru*, adapted to the project's beekeeping domain.

F.1 The five SOLID principles

Principle	Before (initial design)	After (refactored)
S: Single responsibility	The `Ruche` entity carries the calculation method `getZummHoneyActualQuantity()`: it mixes state (data) and business logic. `Visite` groups data and report fields.	The calculation is extracted into `ServiceMielImpl` (via the Composite) and the report becomes `RapportVisite`. Each class now has only a single reason to change.
O: Open / closed	Alert detection and unit conversion would rely on `if / switch` on `typeSeuil` and `unite`: adding a rule or a unit requires modifying existing code.	`RegleAlerte` (Strategy) and `ConvertisseurUnite`: a new rule or unit = a new class, without touching the `GestionnaireAlertes`. Open to extension, closed to modification.
L: Liskov substitution	The use-case diagram generalises the supervising agent from the beekeeping agent. Transposed as is into class inheritance, a restrictive subtype would break substitutability.	`Corps` and `Hausse` inherit from `Compartiment` while fully respecting its contract (capacity 1..10, `getPoidsMiel`). The agent's role goes through the `Role` enumeration + permissions (composition) rather than a restrictive inheritance.

Principle	Before (initial design)	After (refactored)
I: Interface segregation	A single "catch-all" service (CRUD + visits + dashboards + honey API) would force the third-party application to depend on unused methods.	Targeted interfaces: `ServiceRequeteMiel` (the third-party API sees only `getZummHoneyActualQuantity`), `ServiceVisite`, `ServiceTableauDeBord`. Each client depends only on its contract.
D: Dependency inversion	The business layer (EJB) calls JDBC directly (concrete implementation): strong coupling to the database and to `ResultSet`.	The business layer depends on abstractions `RucheRepository` / `VisiteRepository` (DAO pattern), implemented by `RucheRepositoryJdbc`. Mock testing and JDBC → Spring Data JPA migration facilitated (cf. appendix E).

F.2 Chosen design patterns

The following table lists the applied patterns, their category (creational, structural, behavioural) and the layer concerned, with the *before / after* justification.

Pattern	Before	After
Composite ***structural: business***	The honey calculation manually traverses the frame collections with duplicated summation code.	`ElementRuche.getPoidsMiel()` is resolved recursively (Ruche → Compartiment → Cadre): uniform processing of the tree.
State ***behavioural: business***	The hive's status would be tested by scattered `switch` (created, active, harvesting, closed, ...).	`EtatRuche` and its concrete states encapsulate the allowed transitions. Direct alignment with the state-machine diagram.
Strategy ***behavioural: business***	The alert rules, unit conversion and ROI would be hard-coded.	Families of interchangeable algorithms (`RegleAlerte`, `ConvertisseurUnite`), injectable and testable in isolation.
Observer ***behavioural: business***	The measurement code would call the dashboards directly (strong coupling).	`GestionnaireAlertes` notifies the subscribed `Observateur` (alerts dashboard, manager notification) when a threshold is crossed.
Command ***behavioural: control***	The field operations (harvest, split, requeening) are direct calls, not replayable offline.	Each operation becomes a `CommandeVisite`. The `FileCommandesHorsLigne` replays the commands on reconnection (offline mode + deferred synchronisation).
Factory Method ***creational: business***	The hives are created by scattered `new` with manual initialisation of the compartments.	`FabriqueRuche.creer(modele)` produces a preconfigured hive according to the model (Dadant, Langstroth...) with consistent body and capacities.

Pattern	Before	After
Builder *creational: presentation*	The visit report, with its many optional fields, would require a telescoping constructor or scattered setters.	`ConstructeurRapport` builds `RapportVisite` fluently (findings, actions, photos, assessments).
Adapter *structural: data*	The heterogeneous sensors (different formats and units) would be handled case by case.	`AdaptateurCapteur` standardises each source into a `Mesure` with configurable units (automated part).
Facade *structural: control*	The Servlet orchestrates all the business services directly.	`FacadeApplication` offers a simplified entry point to the Servlets on top of the subsystems.
Singleton *creational: cross-cutting*	The `ConfigZumm.ini` file and the i18n labels would be re-read in several instances.	`Configuration` and `GestionnaireI18n` guarantee a single shared instance.
Proxy *structural: web services*	Access to the API would directly expose the business service.	A `ProxySecurite` controls OAuth authentication and SSL / X.509 encryption before delegating to the service.

F.3 Refactored design view

Figure F.1 summarises the main patterns in a design class diagram: the Composite (hive composition and honey calculation), the State (life cycle), the segregated services (ISP), abstraction-based persistence (DIP / Repository), the Strategy + Observer pair (alerts) and the Factory / Builder / Command trio.

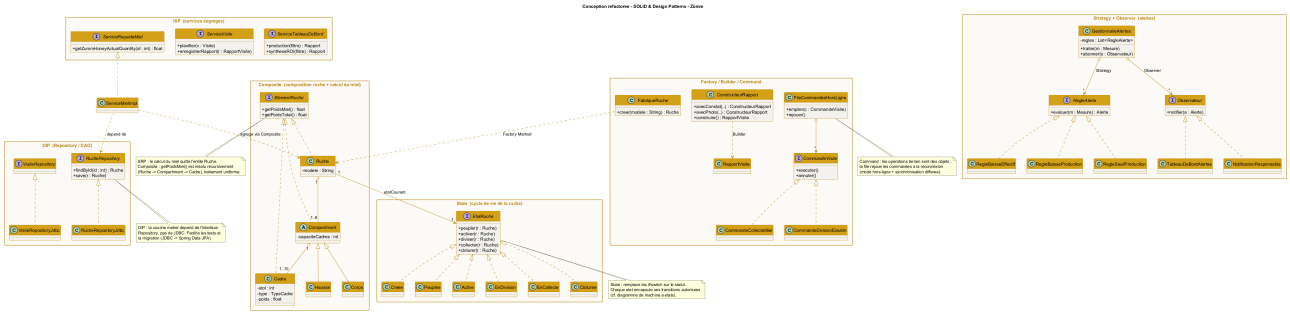

Figure F.1. Refactored class diagram: application of SOLID and the design patterns to the Zümm system.

Correspondence with the layers: the `Repository` belong to the data-access layer (JDBC), the `Service` and the domain (Composite, State, Strategy, Observer, Command) to the business layer (EJB), the `Facade` and the `Proxy` to the control layer (Servlets), the report `Builder` to the presentation (JSP). The loosely coupled layered architecture of chapter 7 is thus reinforced, without being modified.

F.4 Dynamic illustration: Observer and Command patterns

Two sequence diagrams detail the behaviour of the system's most structuring patterns.

F.4.1 Observer (+ Strategy): triggering an alert

Figure F.2 shows how a measurement received by the `GestionnaireAlertes` (subject) is evaluated by an interchangeable rule (*Strategy*) then, in case of a breach, propagated to the subscribed observers (alerts dashboard, manager notification). Adding a rule or an observer requires no modification of the manager (open/closed principle).

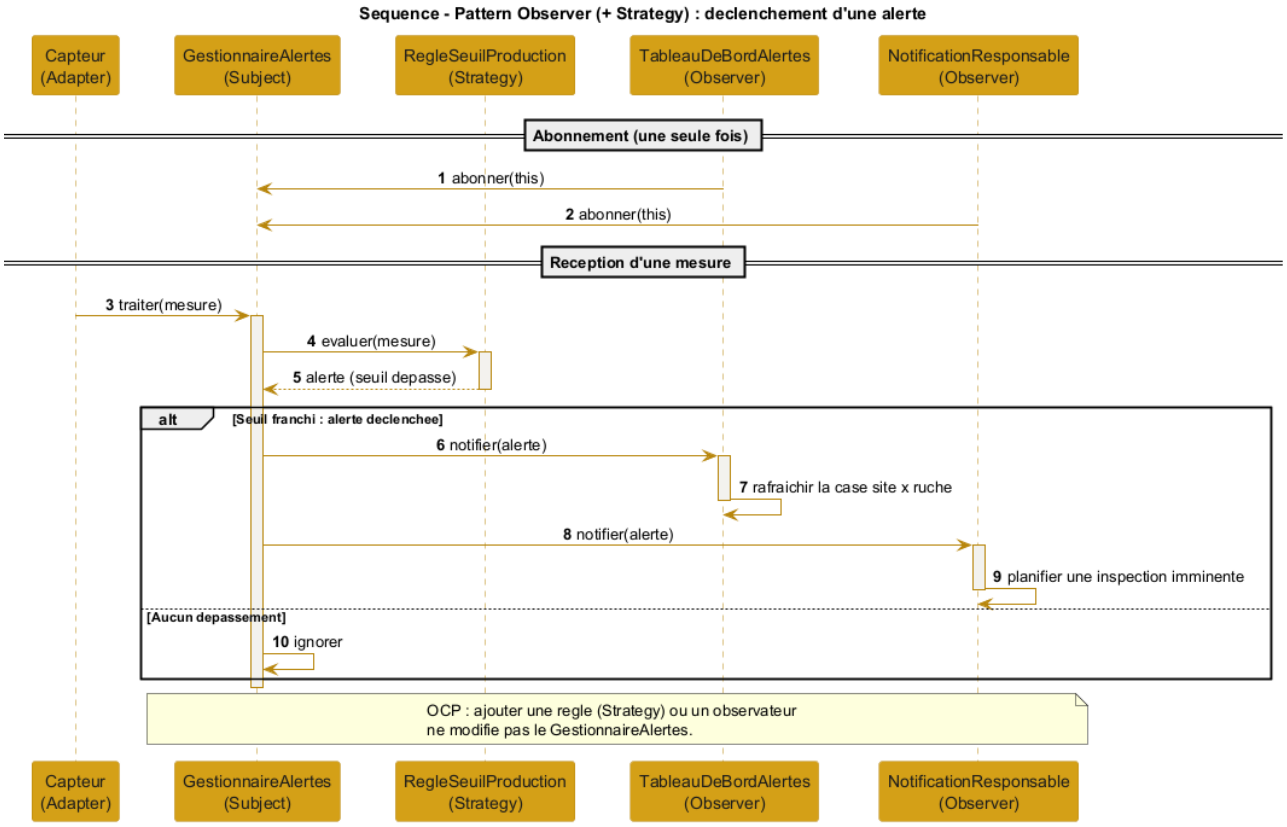

Figure F.2. Sequence of the Observer (+ Strategy) pattern: alert-triggering flow.

F.4.2 Command: offline mode and deferred synchronisation

Figure F.3 illustrates the entry of a field operation without a network: the operation is encapsulated in a `CommandeVisite` pushed onto the `FileCommandesHorsLigne`. When the network returns, the queue replays (`executer`) each command, or compensates it (`annuler`) in case of conflict, achieving the deferred synchronisation required by the offline mode.

F.4.3 State: hive life cycle

Figure F.4 shows the Ruche (context) delegating each operation to its current state. The state decides the transition (activation, harvesting, return to the active state) or refuses an operation forbidden in the current state, here a populating requested on an already active hive. The behaviour thus depends on the state without any `switch` on the status, consistently with the state-machine diagram (figure 7.8).

F.4.4 Composite: honey-quantity calculation

Figure F.5 details the recursive resolution of `getPoidsMiel()`: the client (`ServiceMielImpl`) queries the hive, which propagates the call to each compartment, then to each frame (leaf). Only frames of

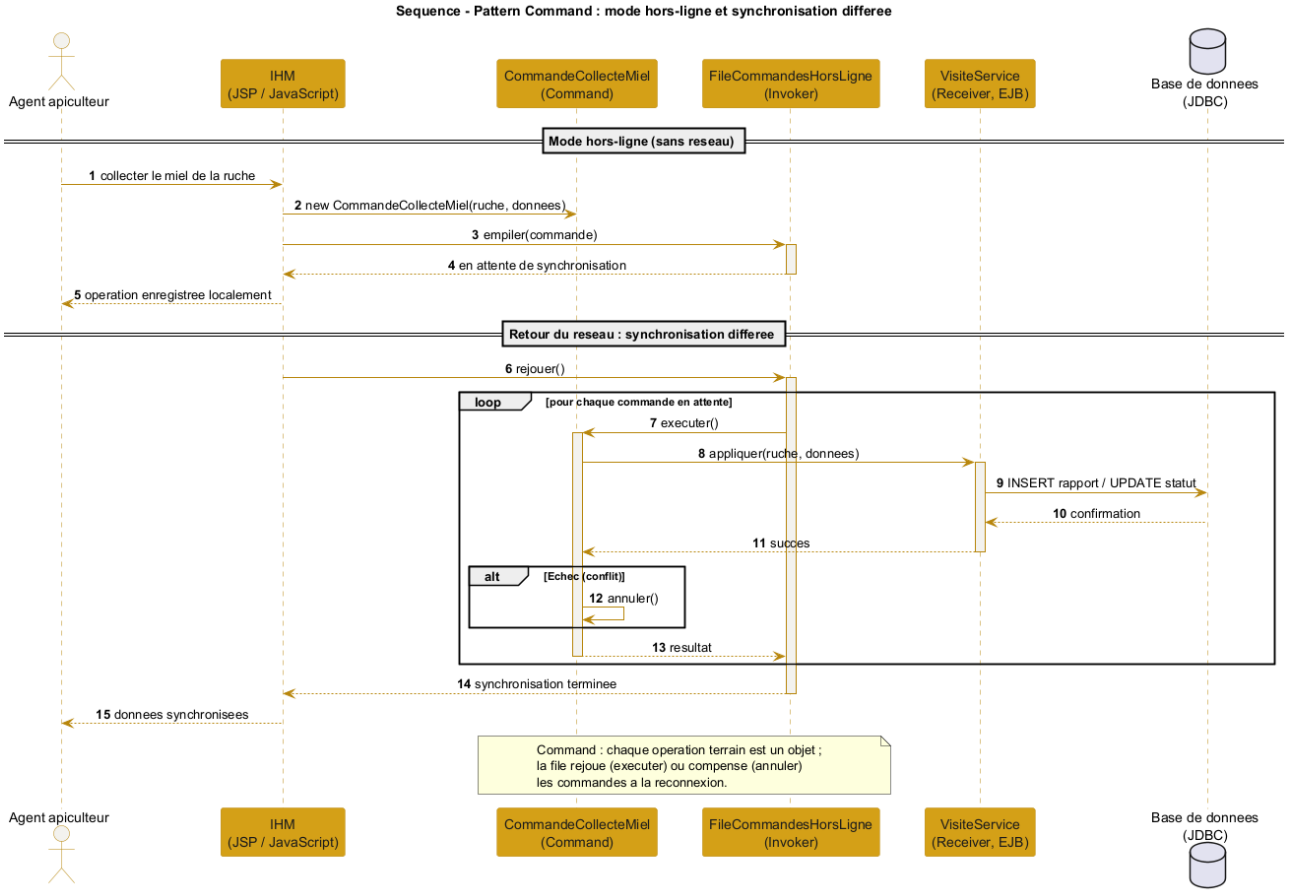

Figure F.3. Sequence of the Command pattern: offline mode and deferred synchronisation.

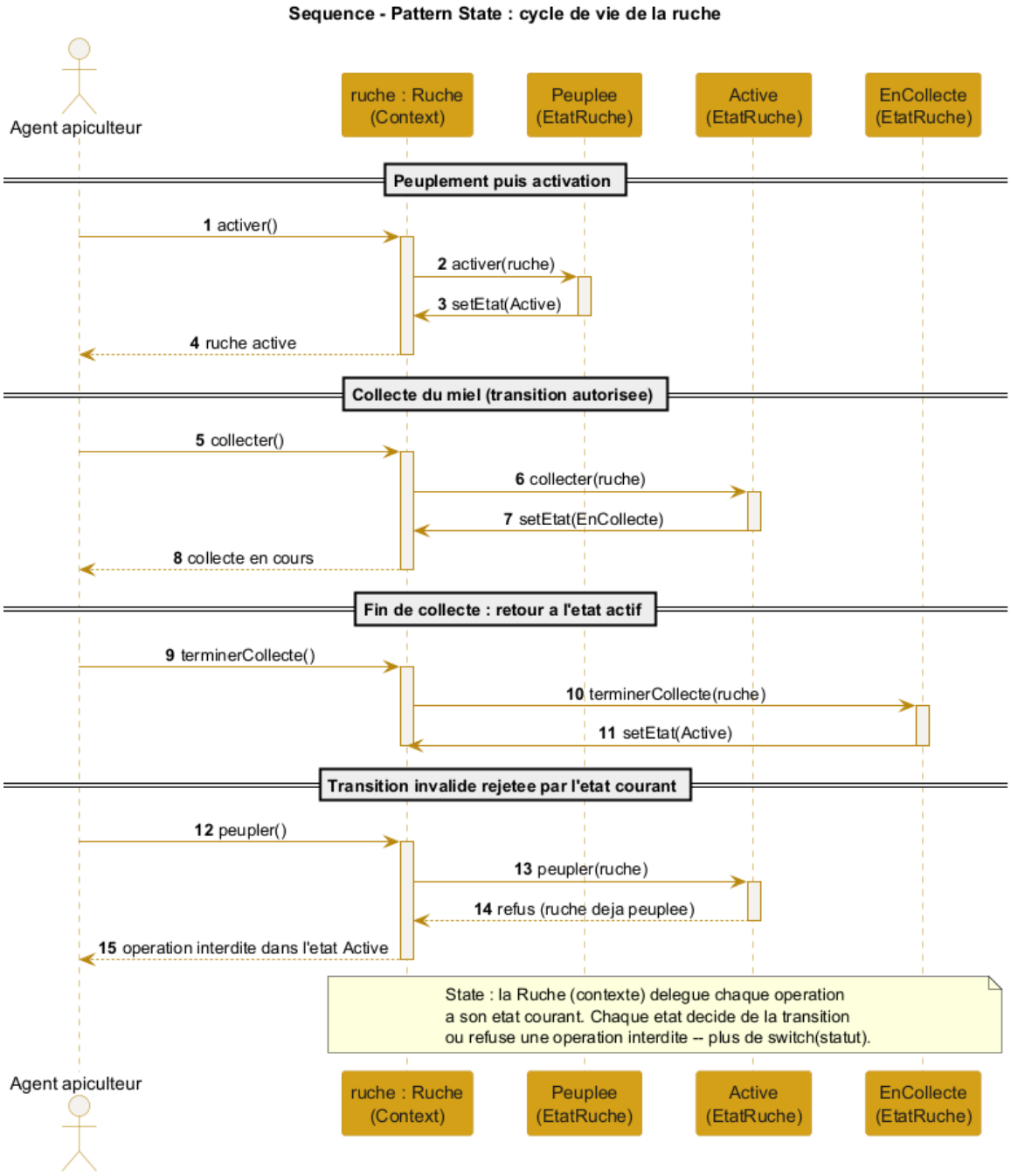

Figure F.4. Sequence of the State pattern: delegation of the transitions of the hive life cycle.

type `MIEL` contribute, and the tare is subtracted at the hive level. The client ignores the internal structure. This calculation implements the `getZummHoneyActualQuantity` method exposed by the web service.

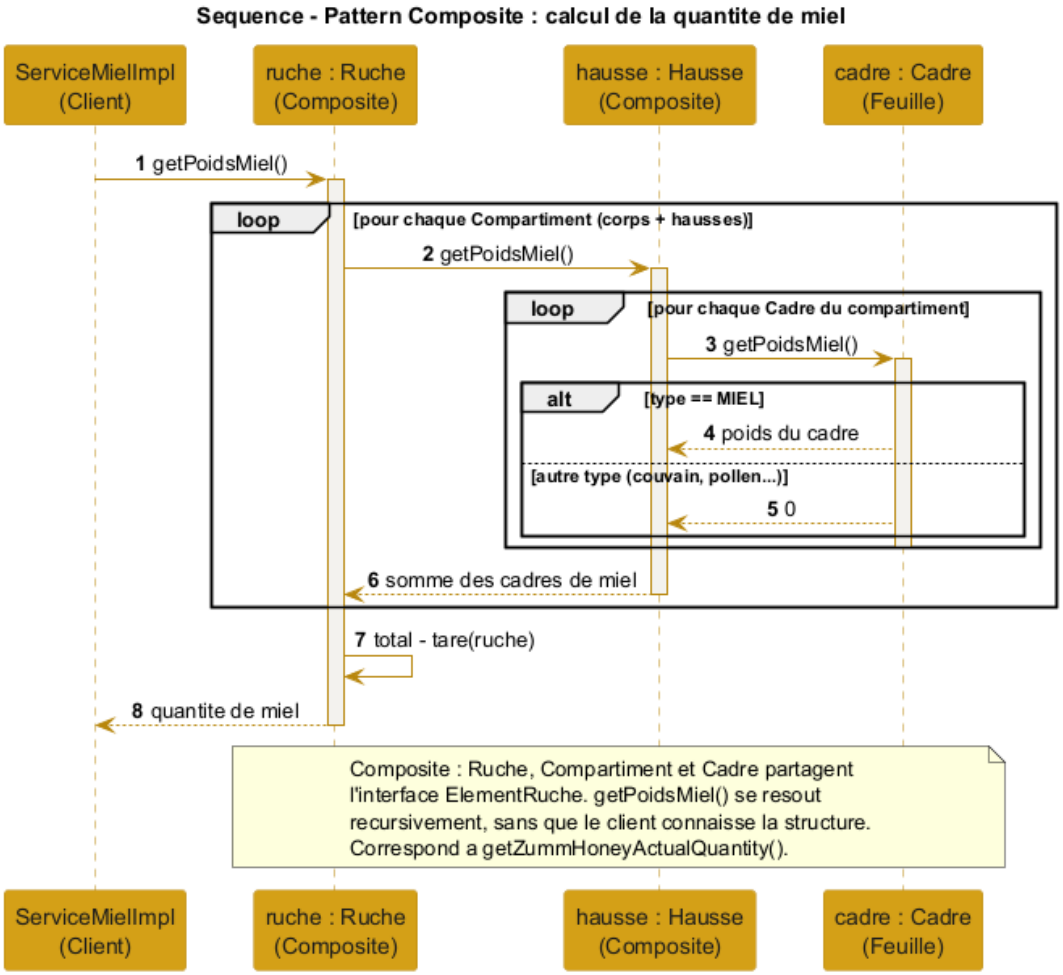

Figure F.5. Sequence of the Composite pattern: recursive calculation of a hive's honey quantity.

F.4.5 Strategy: conversion of heterogeneous units

Figure F.6 shows the normalisation of a measurement expressed in any unit: the `RegistreConvertisseurs` (context) selects the strategy corresponding to the unit (`ConvertisseurLbVersKg`), which brings the value back to the reference unit before comparison with the threshold. A new unit translates into a new strategy, without modifying the context.

F.4.6 Adapter: ingestion of heterogeneous sensors

Figure F.7 illustrates the integration of a market sensor: the `AdaptateurBroodMinder` translates the sensor's proprietary API into a standardised `Mesure` (converted units, timestamp, geolocation), exposing the uniform `Capteur` interface expected by the business layer. The heterogeneous sensors thus become interchangeable.

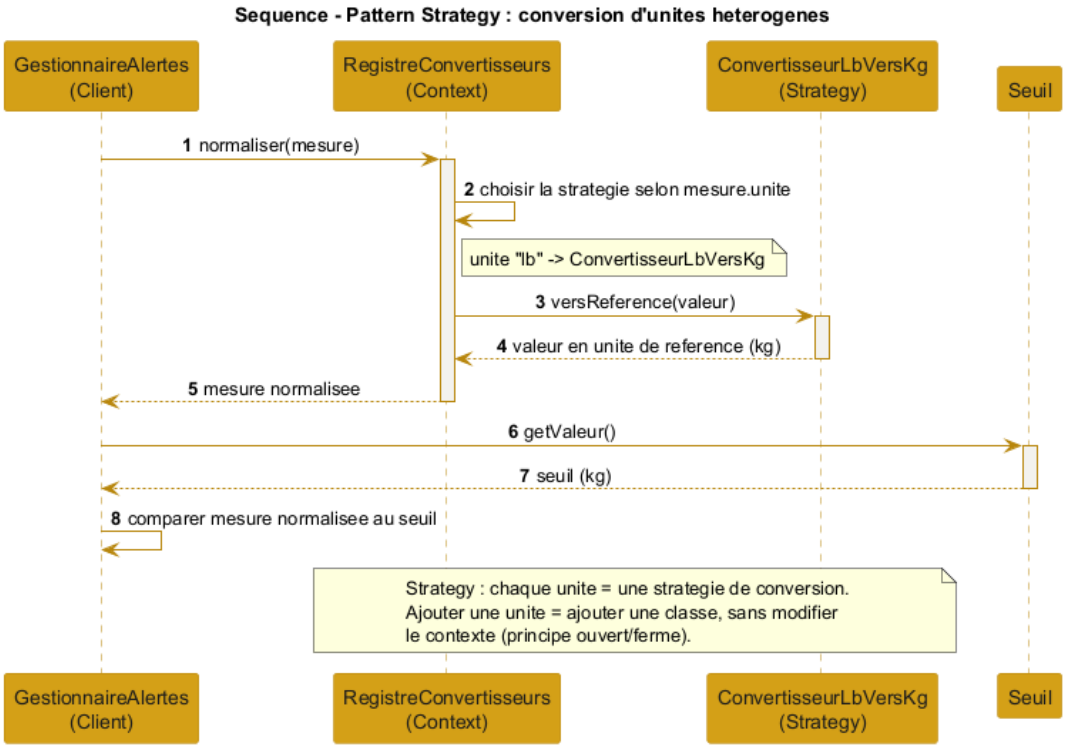

Figure F.6. Sequence of the Strategy pattern: conversion of heterogeneous units to the reference unit.

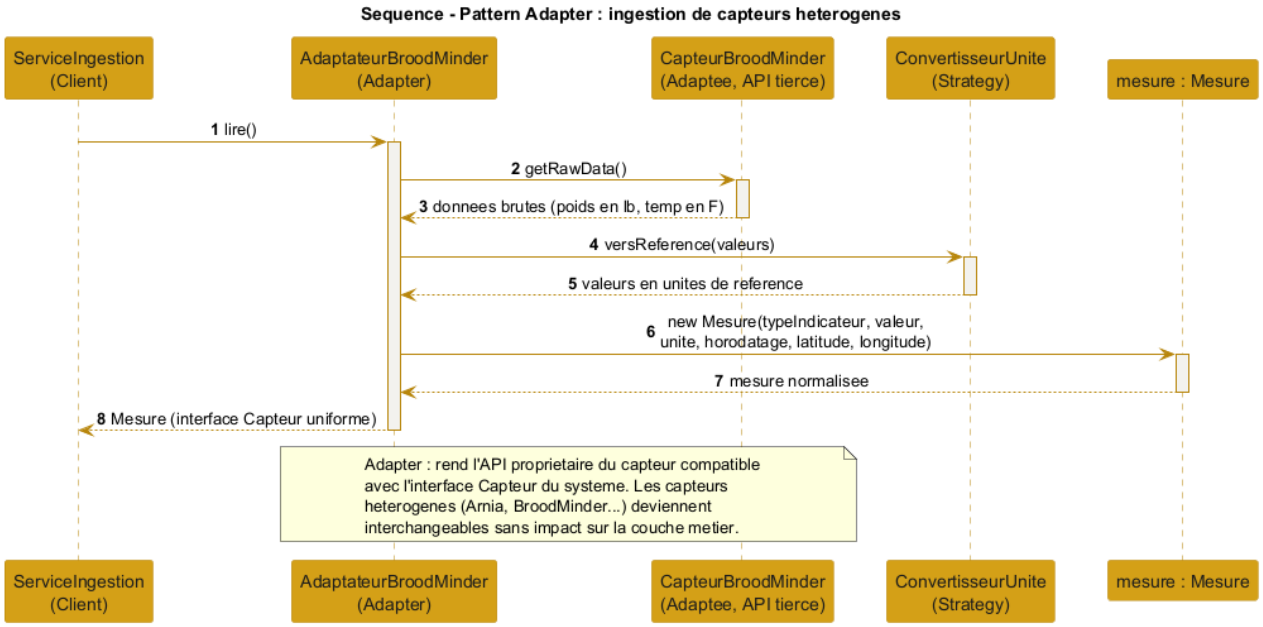

Figure F.7. Sequence of the Adapter pattern: ingestion of heterogeneous sensors (automated part).

F.5 Distribution of the patterns per layer

Figure F.8 summarises the location of each pattern in the multi-tier architecture. The *Builder* sits in presentation. The *Facade*, the *Proxy* and the *Command* queue belong to control. The domain (*Composite*, *State*, *Observer*, *Strategy*, *Factory Method*) and the segregated services (*ISP*) are placed in business. The *Repository* (*DAO / DIP*) and the *Adapter* handle data access. Finally, the configuration and internationalisation *Singleton* as well as the unit-conversion *Strategy* occupy the cross-cutting layer. This view confirms that the patterns reinforce the loosely coupled layered architecture without altering its structure.

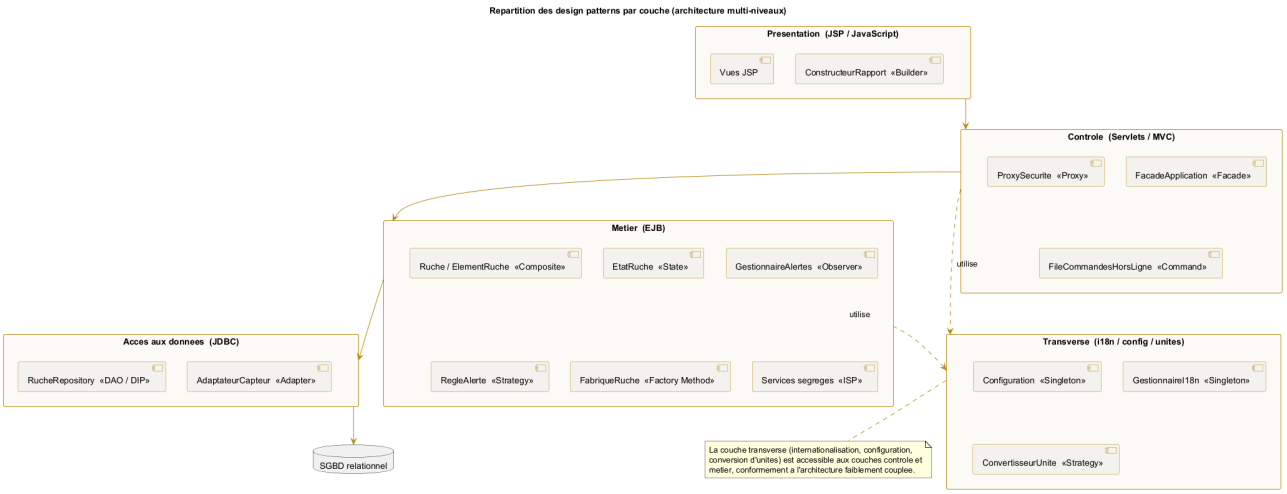

Figure F.8. Distribution of the design patterns per layer of the multi-tier architecture.

APPENDIX G

Appendix – Complements: data, UI, security and tests

This appendix gathers complementary deliverables reinforcing the file: the logical data model, the interface mockups, the access-rights matrix, the risk register, data-protection compliance and the test strategy.

G.1 Logical data model (LDM)

The LDM translates the data dictionary (chapter 5) into relational tables with primary keys, foreign keys and management constraints (`CHECK` on the number of supers, the frame capacity and the productivity, as well as the uniqueness of the compartment/slot pair). It distinguishes the *location* relationship (a site hosts hives) from the *ownership* relationship (a farm owns hives), in accordance with the rule allowing a site to host hives from several farms.

G.1.1 Vocabulary correspondence table

To guarantee consistency between the deliverables, each business entity carries the same concept in three forms: the **data dictionary** name (chapter 5), the UML diagram **class** and the LDM **table**. Attribute writing convention: `camelCase` in the classes (`nbHausses`), `snake_case` in the tables (`nb_hausses`).

Table G.1. Name correspondence between dictionary, classes and LDM.

Dictionary (business)	Class (UML)	Table (LDM)
Farmer	`Fermier`	`FERMIER`
Farm	`Ferme`	`FERME`
Site	`Site`	`SITE`
Hive	`Ruche`	`RUCHE`
Compartment (body / super)	`Compartiment`	`COMPARTIMENT`
Frame	`Cadre`	`CADRE`
Agent	`Agent`	`AGENT`
Schedule	`Planning`	`PLANNING`
Visit	`Visite`	`VISITE`
Photo	(visit report)	`PHOTO`

Dictionary (business)	Class (UML)	Table (LDM)
Measurement	`Mesure`	`MESURE`
Harvest	`Recolte`	`RECOLTE`
Threshold	`Seuil`	`SEUIL`

G.2 Interface mockups (wireframes)

The following low-fidelity mockups set the ergonomics of the key screens before development. They respect the visual identity and the required simplicity.

G.3 Access-rights matrix (RBAC)

The matrix specifies, for each function, the right granted to each profile. **Legend:** CRUD = create / read / update / delete, R = read, X = execute, A = approve, — = no access.

Function	Owner	Beek.	Sup.	Mgr.	Admin	API
Farms and sites	CRUD	R	R	R	CRUD	—
Hives (bodies/frames)	CRUD	R	R	R	CRUD	—
Agents	R	—	R	R	CRUD	—
Plan a visit	R	X	X	X	X	—
Approve the schedule	—	—	A	—	X	—
Carry out visit/report	R	X	X	—	R	—
Agents x hives calendar	R	R	R	R	R	—
Production dashboard	R	—	—	R	R	—
Alerts dashboard / handle	R	—	—	X	R	—
Summary and ROI	R	—	—	R	R	—
Configure thresholds/units	—	—	—	R	CRUD	—
Internationalisation	—	—	—	—	CRUD	—
CSV / TXT export	R	R	R	R	R	—
Backup / restore	—	—	—	—	X	—
Honey-quantity API	—	—	—	—	—	R

This matrix must be applied server-side (systematic rights checking in the control layer), and not only by hiding the interface elements.

G.4 Risk register

Risk	Prob.	Impact	Reduction measure
Development delay	Medium	High	Prioritise the CRUD core, with weekly milestones (W5 to W12).
Scope too broad	Medium	High	Limit to the high-priority functions, the rest optional.

Risk	Prob.	Impact	Reduction measure
Arabic i18n complexity (RTL)	Medium	Medium	Early end-to-end FR/EN/AR testing, `dir=rtl` handling.
Loss of field data	Medium	High	Offline command queue + synchronisation + backup.
Application vulnerabilities (injection, XSS)	Medium	High	`PreparedStatement`, JSP escaping, security review.
False alerts (sensors)	Medium	Medium	Configurable thresholds + human validation through the report.
Google OAuth unavailability	Low	Medium	Local authentication fallback for the demonstration.
Measurement volume	Low	Medium	Archiving and aggregation per period (automated part).
Use of a code generator	Low	Very high	In-house design and code, traceability and peer review.

G.5 Protection of personal data

The system handles personal data (names and contacts of the farmers and agents) and potentially sensitive data (precise **geolocation** of the sites). The following principles are adopted, in the spirit of the GDPR:

- **Minimisation and purpose**: collect only the data useful for beekeeping monitoring.
- **Security**: encryption of exchanges (SSL / X.509) and authenticated access, already planned.
- **Access control**: strict application of the RBAC matrix above.
- **Individuals' rights**: viewing, rectification and **erasure** of the data, **portability** ensured by the CSV / TXT export.
- **Retention**: retention period defined for the measurements and visits, beyond which the data are archived or anonymised.
- **Traceability**: logging of access and sensitive actions (schedule approval, site closure).

The above principles translate into the following technical measures, by data category. **Encryption at rest** targets, as a priority, the sensitive columns (contacts, geolocation). **Pseudonymisation** dissociates the agent's identity from the agronomic history upon anonymisation.

Table G.4. Technical data-protection measures (spirit of the GDPR).

Data	Purpose	Retention	Security / erasure
Identity (name, contact)	Accounts and responsibilities	Relationship + 1 year	Encryption at rest, RBAC, anonymisation on request
Site geolocation	Monitoring, maps, weather	Site lifetime + 1 year	Encryption at rest, reducible precision, erasure at closure
Visits and reports	Health and production traceability	5 years	Encryption at rest, pseudonymisation (agent dissociation)

Data	Purpose	Retention	Security / erasure
Inspection photos	Visual monitoring of the colony	2 years	Encrypted storage, restricted access, deletion with the report
Sensor measurements	Performance analysis	Raw 1 year, then aggregated	Encryption at rest, aggregation / anonymisation
Access logs	Security and accountability	6 to 12 months	Administrator access, automatic purge

Erasure procedure: on request from a concerned individual, the system first produces an export (CSV / TXT portability), then erases or anonymises the records according to the table above. A **record of processing activities** and a data-protection referent are maintained for the project.

G.6 Test strategy

The strategy complements the functional test plan (chapter 10) with the unit and integration levels. The loosely coupled design (`Repository` interfaces, segregated services) makes the business rules testable in isolation, by injecting doubles (mocks).

Level	Scope	Examples	Tool
Unit	Isolated management rules	Frame/super constraints, honey calculation (Composite), ROI, thresholds	JUnit 5, Mockito
Integration	Data access (Repository/JDBC)	Persisted CRUD, calculation queries	JUnit + H2 database
System	End-to-end functional cases	Cases TC01 to TC16 of chapter 10	JUnit, Selenium
Acceptance	Validation by the demonstration	Agent, farmer, manager journeys	Defence

- **Target coverage**: at least 70 % of the business layer (management rules), measured by `JaCoCo`.
- **Traceability**: each requirement is linked to at least one test case (matrix of chapter 10), completed by the corresponding unit tests.
- **Automation**: execution of the tests at each build (Maven), a prerequisite for delivery.

Synergy with the design: dependency inversion (DIP) makes it possible to substitute an in-memory `RucheRepository` for the JDBC implementations, which makes the tests fast, deterministic and independent of the database.

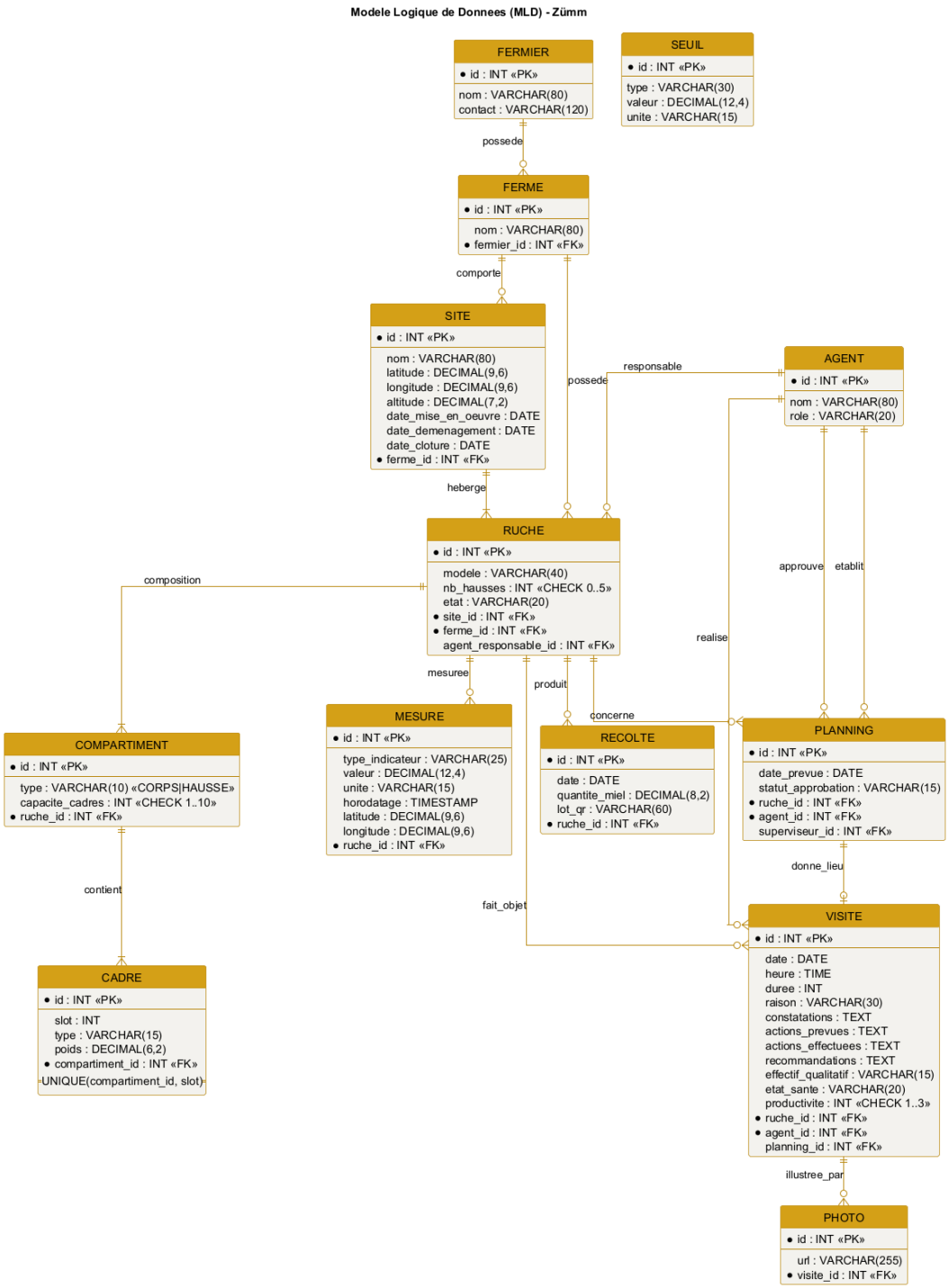

Figure G.1. Relational logical data model of the Zümm system.

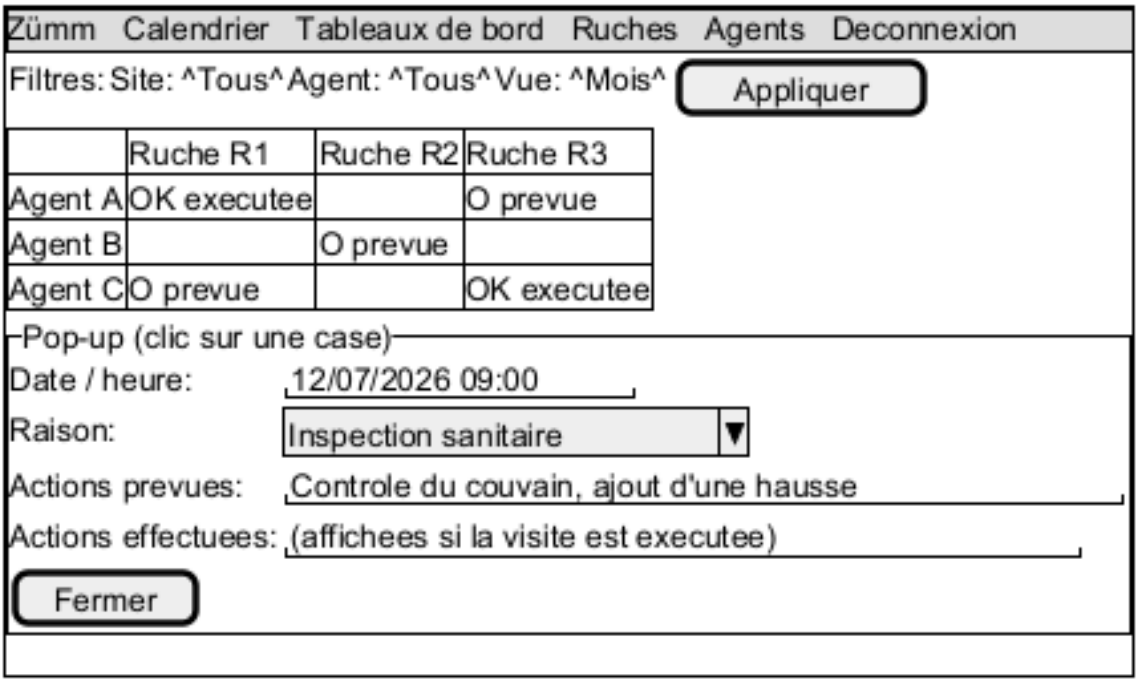

Figure G.2. Mockup: calendar / agents x hives matrix with visit pop-up.

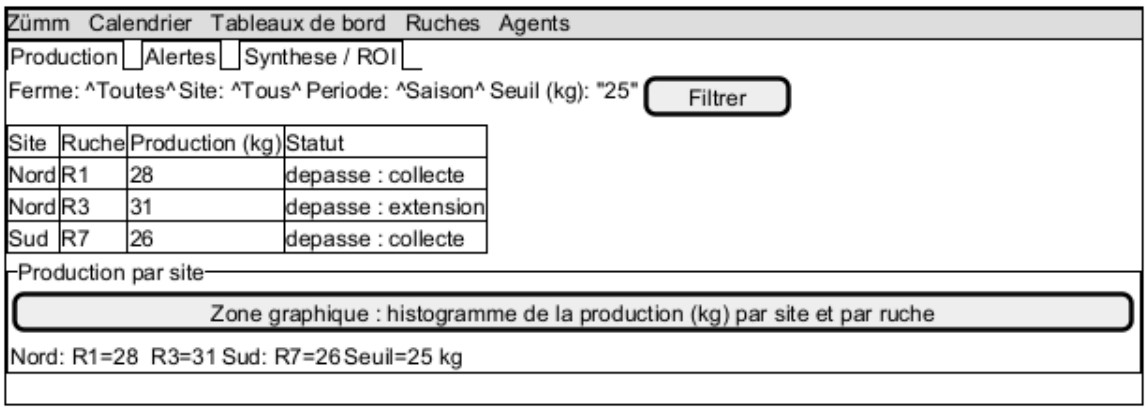

Figure G.3. Mockup: production dashboard with configurable threshold.

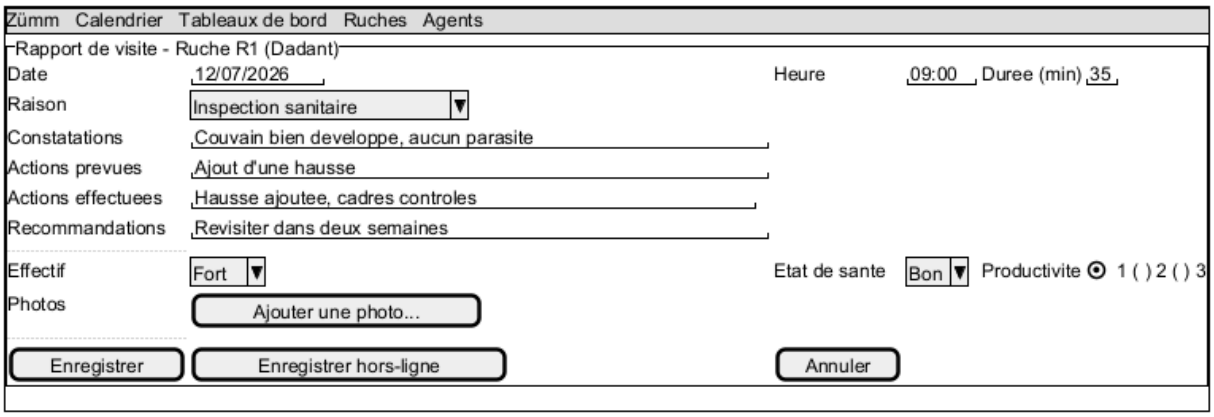

Figure G.4. Mockup: visit-report form (with offline mode and photos).

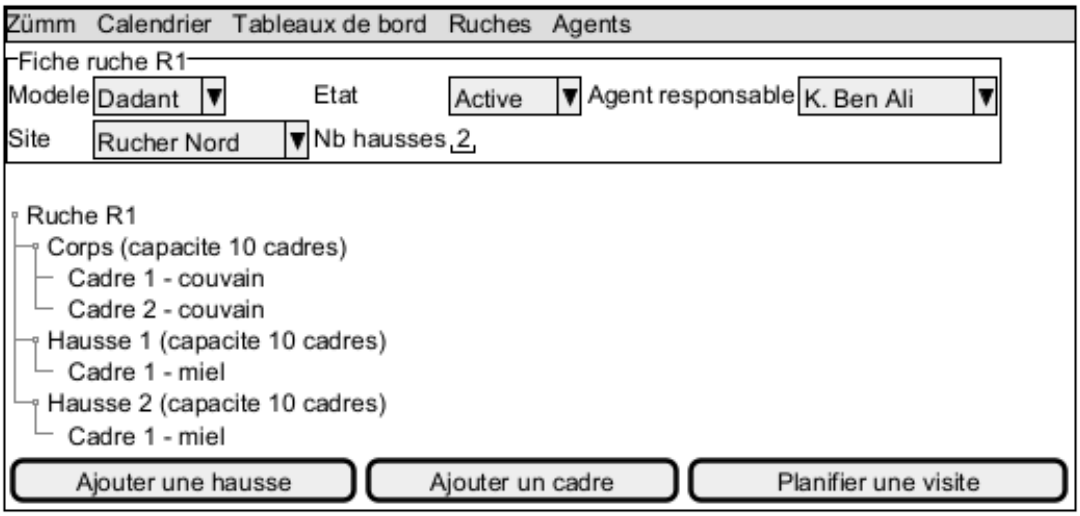

Figure G.5. Mockup: hive form and composition (body, supers, frames).

APPENDIX H

Appendix – Artificial intelligence and predictive analysis

This appendix answers the questions of the *why*, the *how* and the *where* of artificial intelligence in **Zümm**. It creates no new requirement: it extends the sensor part and the predictive analysis already anticipated in chapter 7 (§ 4.5) and the *data* microservice trajectory mentioned in appendix E. The objective is to document an AI integration **consistent with the loosely coupled architecture**, **without betraying the constraints** of the requirements specification (self-deployment, free of charge, human validation).

Positioning: in Zümm, AI is a **decision-support module**, never a control organ. It produces explainable *pre-alerts*; the beekeeping agent keeps the **final validation** through the visit report, in accordance with the evaluation rule of § 4.5.4.

H.1 Guiding principles

Every AI building block adopted respects four principles drawn directly from the project requirements:

- **Decision support and human validation**: AI flags, it does not decide. Each output is confirmed by the agent (§ 4.5.4).
- **Self-deployment without a mandatory third-party service**: no essential function depends on a paid cloud API. The basic processing runs **locally**; cloud use remains *optional* and enabled by configuration.
- **Decoupling and modularity**: AI is a **module enabled** via `ConfigZumm.ini`, isolated from the business core by an interface, in the spirit of loose coupling (appendix F).
- **Explainability and frugality**: **interpretable** methods (statistics, rules) and economical ones are favoured, rather than opaque models, to remain defensible and reproducible.

H.2 Mapping of AI uses

The table below lists the AI uses relevant to Zümm, distinguishing what is **in scope** (implementable in the present foundation) from what is **anticipated** (interface specified, later development under the sensor part).

Use	Description	Cloud dep.	Effort	Status
Adaptive anomaly detection	Per-hive baseline replacing the fixed threshold, to detect an unusual drift in production, population or temperature.	No	Low	**In scope**
Acoustic swarming detection	Analysis of the sound signature for a swarming pre-alert 1 to 5 days in advance (precision beekeeping, § 4.5.1).	No	High	Anticipated
Vision: disease detection	Help in identification (varroa, brood) from the inspection photos attached to the visit report.	No	High	Anticipated
Assisted voice entry	Dictation of the report, transcription feeding the report fields (§ 4.7, low priority).	Optional	Medium	Anticipated
Summary assistant	Natural-language summary of the dashboards (ROI, alerts) for the owner.	Optional	Medium	Optional

Prioritisation recommendation: deliver the **adaptive anomaly detection** as a real function (feasible within the imposed foundation, without cloud dependency), and **specify the other uses as anticipated interfaces**. This is the posture already adopted by the requirements specification for the sensor part, which preserves the document's consistency.

H.3 Selected case: adaptive anomaly detection

§ 4.5.4 defines **fixed thresholds** (for example internal temperature $< 32\,°\text{C}$ or $> 36\,°\text{C}$) and an anti-rebound rule. The limit of a fixed threshold is that it ignores the **specific behaviour of each hive** and its **seasonality**. The natural evolution consists in learning a **per-hive baseline** and measuring the deviation from this baseline, which detects the "relative drop $> 20\,\%$ vs previous period" adaptively.

H.3.1 Formalisation (without code)

For each (hive, indicator) pair, an **exponentially weighted moving average** (EWMA) μ_t and a dispersion estimate σ_t are maintained, updated at each new normalised measurement x_t (reference unit, § 5.3):

$$\mu_t = \alpha\, x_t + (1-\alpha)\, \mu_{t-1}, \tag{H.1}$$

$$v_t = \alpha\, (x_t - \mu_{t-1})^2 + (1-\alpha)\, v_{t-1}, \qquad \sigma_t = \sqrt{v_t}. \tag{H.2}$$

The **anomaly score** is the normalised deviation (sliding *z-score*):

$$z_t = \frac{|x_t - \mu_{t-1}|}{\sigma_{t-1} + \varepsilon}. \tag{H.3}$$

A **pre-alert** is raised only if $z_t > k$ (sensitivity) **persistently over N consecutive measurements** (hysteresis, as in § 4.5.4). This continuity with the existing rule is deliberate: an absolute threshold is

replaced by a *learned relative* threshold, without changing the anti-rebound logic or the human-validation principle.

H.3.2 Parameters (all configurable via ConfigZumm.ini)

Parameter	Role	Effect of the setting
α **(smoothing)**	Weight of the recent measurement	The larger α, the faster the baseline reacts (and the more sensitive to noise).
k **(sensitivity)**	Threshold on the z-score	The smaller k, the more sensitive the detection (and the more false alerts increase).
N **(persistence)**	Number of consecutive measurements	The larger N, the fewer false alerts (at the cost of a slight delay).
Window / season	Reference history	Allows a per-season baseline (nectar flow, wintering).

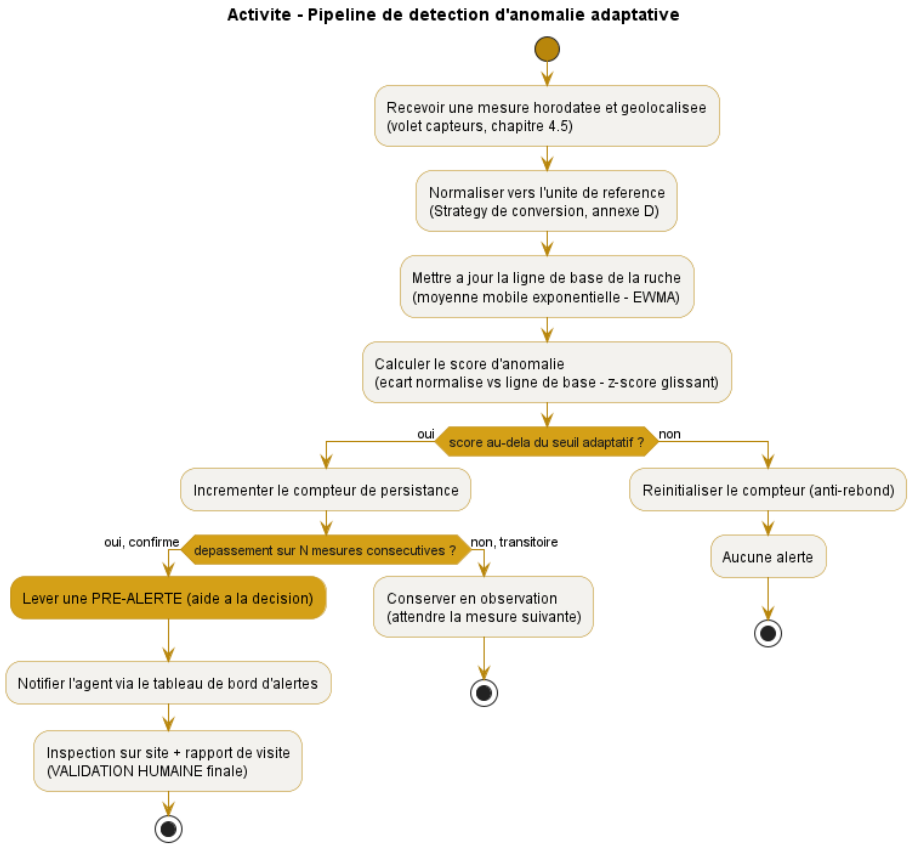

Figure H.1. Adaptive anomaly-detection pipeline, from the raw measurement to the pre-alert confirmed by the agent.

Consistency with § 4.5.4: the value is first **converted to the reference unit** (conversion Strategy, appendix F), then compared with a threshold; the alert remains subject to **anti-rebound**

and remains a **decision-support aid**. Only the nature of the threshold changes: from *fixed* to *learned per hive*.

H.4 Integration architecture

AI fits in without breaking anything in the layered architecture (figure 7.2). Two decoupling mechanisms are used.

H.4.1 The detector as an interchangeable Strategy

Anomaly detection becomes a **`RegleAlerte` strategy** (Strategy pattern, appendix F), in the same way as the existing fixed threshold. The `GestionnaireAlertes` chooses, by configuration, between "fixed threshold" and "adaptive baseline" without its code changing (open/closed principle). AI is therefore not an invasive addition, but an **additional implementation of an already planned interface**.

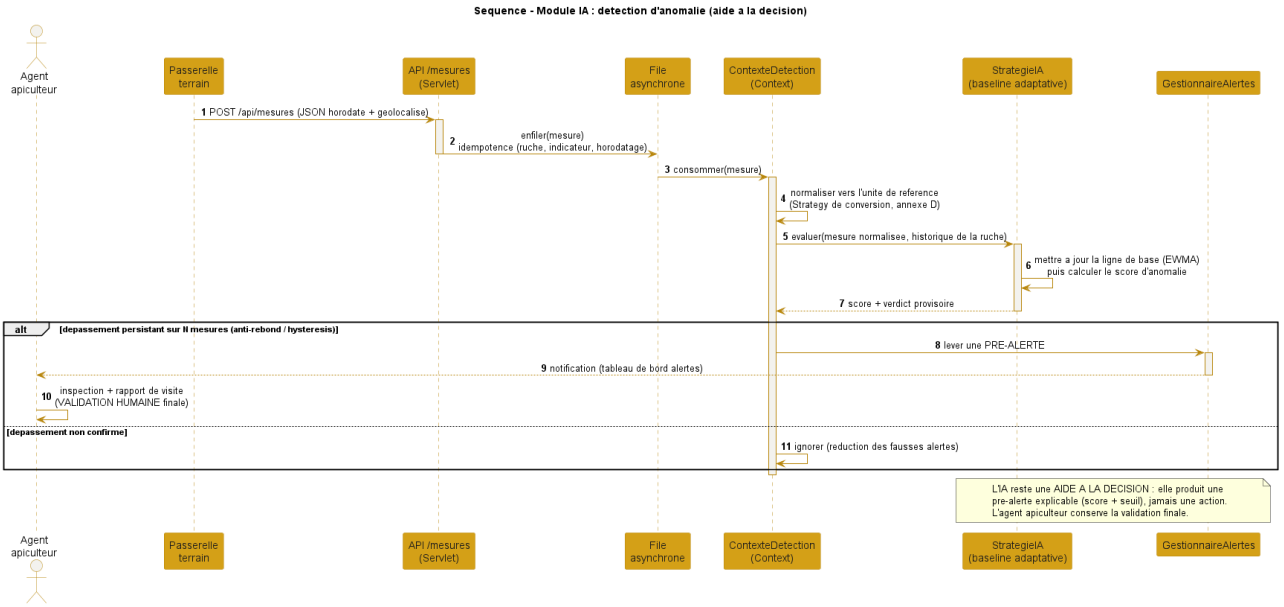

Figure H.2. Sequence of the AI module: the ingested measurement is evaluated by the adaptive strategy, which raises a pre-alert confirmed by the agent.

H.4.2 The heavy processing as a decoupled microservice

The uses requiring **signal or vision** (acoustics, photos) must **never** run in the EJB container. They are offloaded to a **Python microservice** (pandas, scikit-learn / TensorFlow), exposed to the Java core by **REST/JSON**, exactly the *data* trajectory described in appendix E. This microservice is **optional**: in its absence, the manual part and the basic statistical detection work fully.

H.5 Anticipated uses (specified interfaces, out of development scope)

In accordance with the treatment of the sensor part, the advanced uses are **specified as interfaces** so that the data model and the architecture accommodate them, without being developed in the present project.

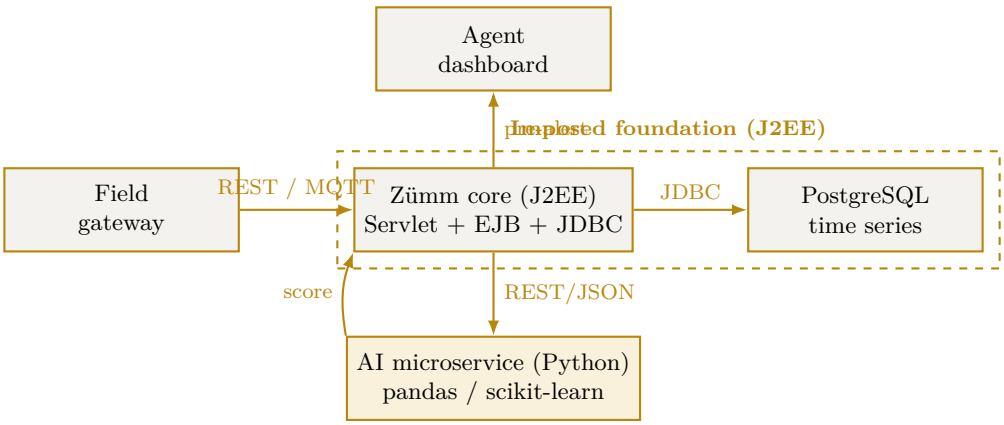

Figure H.3. Decoupled integration: the J2EE foundation remains in control, the AI microservice is an optional module plugged in via a web service.

- **Acoustics / swarming**: the data model already accommodates a `SIGNATURE_ACOUSTIQUE` indicator (§ 4.5). The AI microservice returns a verdict (swarming pattern detected, queen presence) that feeds a pre-alert.
- **Vision / diseases**: the **inspection photos** already attached to the visit report constitute the input set; a pre-trained model returns a suggestion, validated by the agent.
- **Voice entry**: the transcription can rely on the **browser's Web Speech API** (free, client-side, without server dependency), with an *optional* cloud fallback enabled by `ConfigZumm.ini`.

H.6 Ethics, limits and compliance

Issue	Adopted provision
False alerts	Hysteresis over N measurements and adjustable sensitivity k; the alert remains indicative.
Over-reliance on automation	Explicit positioning as decision support; the agent keeps the final validation (§ 4.5.4).
Explainability	Interpretable methods (score + threshold) preferred to opaque models; the pre-alert displays its justification.
Data protection	Local processing by default, SSL / X.509 encryption of exchanges, no imposed cloud transfer.
No lock-in	The AI microservice is optional and replaceable; without it, the basic management and detection remain operational.
Frugality	Priority to lightweight statistical processing; heavy inference is triggered only on demand.

Conclusion: Zümm's AI boils down to a clear rule — *learn each hive's normal, flag the deviation, let the human decide.* This approach adds predictive value while fully respecting the constraints of self-deployment, loose coupling and human validation of the requirements specification. It turns the *anticipated* predictive part into a **concrete and progressive** implementation trajectory.

APPENDIX I

Appendix – Security and robustness

Security is a **cross-cutting** requirement already present in the requirements specification: encryption of exchanges by **X.509 / SSL**, delegated authentication by **Google OAuth**, the **RBAC** rights matrix and **GDPR** compliance (appendix G). This appendix **consolidates** it into a coherent **defence-in-depth** strategy, then addresses **robustness** (resistance to load and to peaks), since *availability* is one of the three security properties. The good practices adopted follow the domain's reference guides (web-site hardening) and the free **k6** tooling for load testing.

CIA triad: security aims at **Confidentiality** (encryption, access control), **Integrity** (input validation, prepared statements, backups) and **Availability** (anti-DDoS, rate limiting, load tests). No building block is a *mandatory* third-party service: each has a free, self-deployable equivalent (ModSecurity, Let's Encrypt, k6), in line with the self-deployment requirement.

I.1 Synthetic threat model

The table sets the usual risks of a web application against the measure adopted in Zümm.

Threat	Measure adopted in Zümm
Interception of exchanges	End-to-end TLS / SSL, X.509 certificates, HTTPS redirection and HSTS header.
Credential theft	Google OAuth (no stored password), two-factor authentication handled by the provider.
SQL injection	Prepared statements (*PreparedStatement*) via JDBC, input validation and typing.
XSS (injected script)	Systematic output escaping, `Content-Security-Policy` header.
CSRF (forged request)	Per-session anti-CSRF token, `SameSite` cookies.
Session hijacking	`HttpOnly` and `Secure` cookies, session expiry and rotation (already § 7).
Privilege escalation	**RBAC** access control and least privilege (appendix G).
Denial of service (DDoS)	Rate limiting, web application firewall (WAF), asynchronous ingestion of measurements.
Data exposure	Encryption at rest, GDPR minimisation, encrypted backups.

I.2 Defence in depth

Security does not rely on a single barrier but on **successive layers**: if one gives way, the others hold. Figure I.1 projects these layers onto Zümm's architecture.

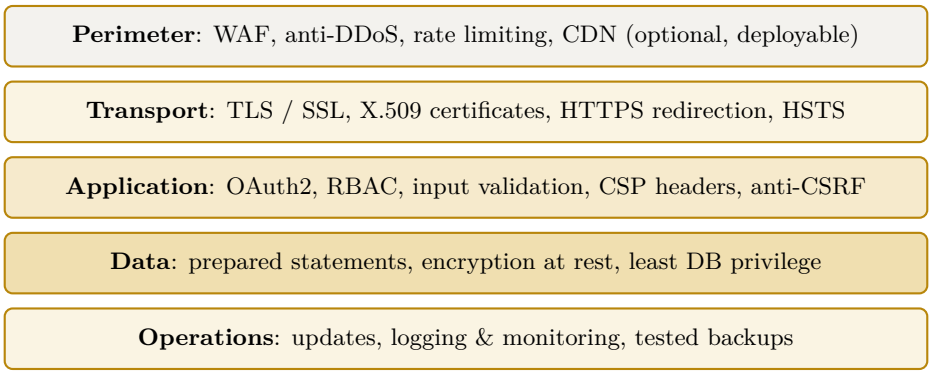

Figure I.1. Defence in depth: five protection layers projected onto Zümm's architecture.

I.2.1 Perimeter and transport

At the edge, a **web application firewall** (WAF, for example ModSecurity) filters malicious requests and **rate limiting** absorbs peaks and abuse attempts. **Transport** encryption is already required: TLS / SSL, **X.509** certificates, forced redirection to HTTPS and the `Strict-Transport-Security` (HSTS) header to forbid any cleartext fallback.

I.2.2 Application

The application layer combines delegated **authentication** (Google OAuth, with two-factor handled by the provider), least-privilege RBAC **access control** (appendix G), **input validation**, **anti-CSRF protection** and **security headers** (`Content-Security-Policy`, `X-Frame-Options`, `X-Content-Type-Options`). Session management relies on `HttpOnly` and `Secure` cookies already specified in chapter 7.

I.2.3 Data and operations

Data access goes exclusively through **prepared statements** (anti-injection), a **least-privilege** database account and **encryption at rest** of sensitive data. In operations, regular **updates** of the server and dependencies, **logging** and **monitoring**, as well as **encrypted backups whose restoration is tested**, close the loop.

I.3 Hardening grid

The table lists web-site hardening good practices and specifies, for each, its **status** in Zümm: already required by the requirements specification or added by this appendix.

Practice	Implementation in Zümm	Status
HTTPS / TLS everywhere	X.509 certificates, HTTPS redirection, HSTS.	Already required

Practice	Implementation in Zümm	Status
Strong authentication	Google OAuth + provider two-factor.	Already required
Access control (RBAC)	Least-privilege roles and permissions.	Already required
Prepared statements	Exclusively parameterised JDBC access.	Already required
Web application firewall (WAF)	Edge request filtering (ModSecurity).	**Added**
Rate limiting / anti-DDoS	Absorption of peaks and abuse.	**Added**
Security headers	CSP, X-Frame-Options, X-Content-Type-Options.	**Added**
Anti-CSRF protection	Per-session token, SameSite cookies.	**Added**
Updates & patches	Up-to-date application server and dependencies.	**Added**
Logging & monitoring	Access and error traces, alerts.	**Added**
Encrypted backups	Regular backup + tested restoration.	**Added**
Anti-malware scanning	Periodic scan of repositories and uploads.	**Added**

I.4 Robustness: load and reliability testing (k6)

Since **availability** is a security property, the system must withstand the nominal load and **peaks**. We adopt **k6**, a **free**, scriptable load-testing tool that simulates *virtual users* (VUs) and checks objective **thresholds**.

I.4.1 Types of tests

Type	Objective
Load	Verify the behaviour at the expected load (nominal users and measurements).
Stress	Find the breaking point by increasing the load beyond nominal.
Spike	Simulate a sudden burst, for example a massive surge of sensor measurements.
Soak (endurance)	Detect memory leaks and degradation over a long duration.

I.4.2 Zümm-specific scenarios

- **Burst ingestion**: `POST /api/mesures` in batches (sensor part, § 4.5), to validate the asynchronous queue and idempotency.
- **Dashboards**: concurrent viewing of the production and alerts dashboards (aggregations, charts).
- **API web service**: repeated calls to `getZummHoneyActualQuantity` by third-party applications.

I.4.3 Thresholds and verdict

The test fails if the objective thresholds are not met, which points to a bottleneck to fix (pool size, index, cache, scaling). For illustration:

```
thresholds:
  request p95 latency        < 500 ms
  HTTP error rate            < 1 %
  /api/mesures ingestion     sustained nominal throughput without loss
```

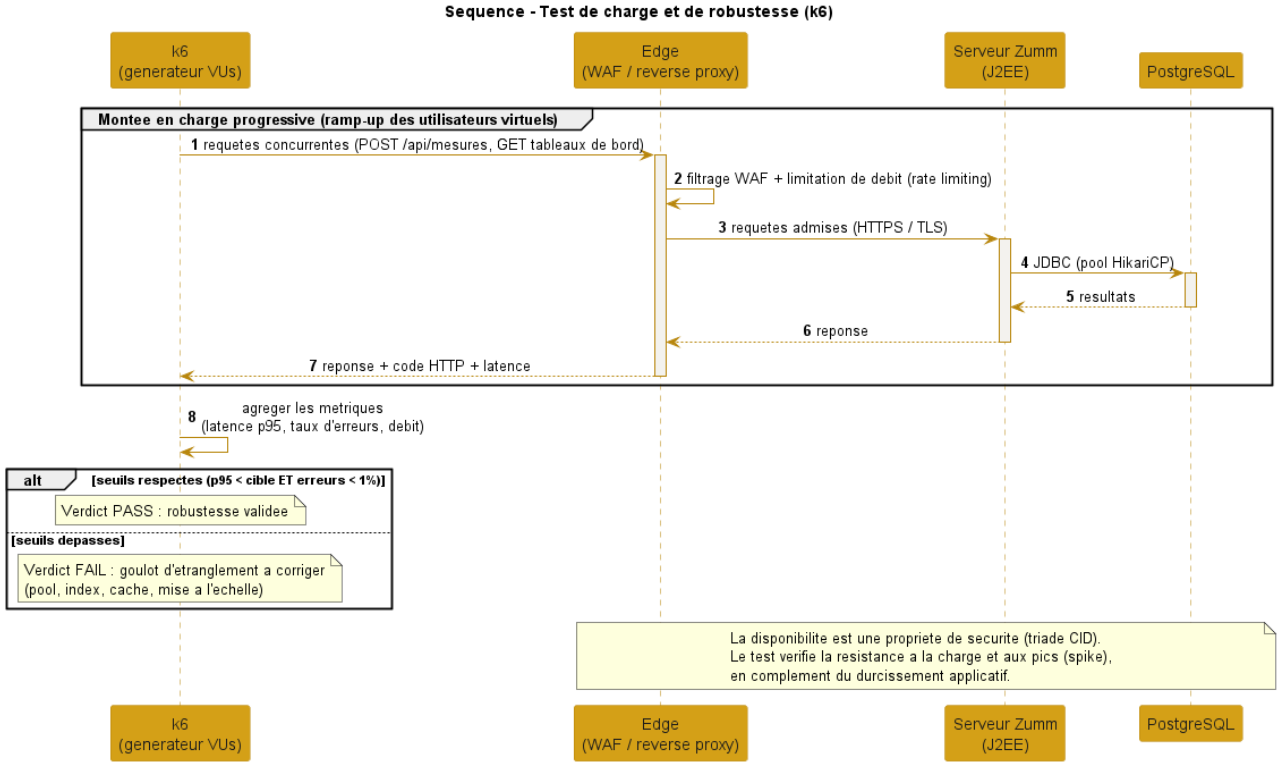

Figure I.2. k6 load test: progressive ramp-up, measurement of latency and error rate, verdict against the thresholds.

I.4.4 Complement to the test plan

The following cases extend the test plan of chapter 11 (TC01–TC16) on the security and robustness aspect.

ID	Object	Expected result
TC17	Load test (k6)	p95 and error-rate thresholds met at nominal load.
TC18	Spike test	The system absorbs a burst without loss or lasting error.
TC19	SQL injection / XSS	Malicious inputs rejected or neutralised (escaping).
TC20	Backup / restoration	Full restoration verified from an encrypted backup.

I.5 Pre-deployment checklist (go-live)

Before any production release, the points below are checked. This checklist summarises the previous sections into an operational review, domain by domain.

Domain	Points to check before production
Secrets & configuration	No cleartext secret in the repository; `ConfigZumm.ini`, keys and certificates out of version control, stored in a protected way.
Transport	HTTPS forced, cleartext-traffic redirection, HSTS header active, X.509 certificates valid and not expired.
Authentication & access	Google OAuth operational, RBAC matrix verified, default accounts and access disabled.
Application	Security headers (CSP, X-Frame-Options), anti-CSRF protection, input validation, error messages without information leakage.
Data	Prepared statements everywhere, encryption at rest of sensitive data, recent backup **and tested restoration**.
Dependencies	Application server and libraries up to date, dependency vulnerability scan performed.
Robustness	k6 load test **green** (p95 and error-rate thresholds), spike test passed, ramp-up and rollback plan defined.
Observability	Access and error logging, monitoring and alerts in place, consistent timestamping.
Compliance	GDPR respected (minimisation, retention period, rights), notices and register up to date (appendix G).

Release rule: an unmet point is **blocking** until fixed. The checklist is replayed at each major release, in the spirit of continuous improvement and under **human control**.

I.6 Compliance and consistency with the requirements

Conclusion: the appendix consolidates **layered** security (perimeter, transport, application, data, operations) and adds **measured robustness** through load tests. It extends, without contradicting, the requirements already set (X.509 / SSL, OAuth, RBAC, GDPR) and respects **self-deployment**: each building block adopted (ModSecurity WAF, Let's Encrypt certificates, k6) is **free and open source**. Security remains, like the alerts, a continuous process placed under **human control**.